



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

Tema 60. Sistemas operativos UNIX-LINUX. Fundamentos, administración, instalación, gestión.

Editado por ASTIC en 2017 a partir de los temas numerados anteriormente como:

- **Tema 53. El sistema operativo UNIX – Linux I. Conceptos básicos.**
AUTORES: María Jesús González/ Servando Martínez/ Pedro Pablo Paños
Actualización 2007
- **Tema 54. El sistema operativo UNIX – Linux II. Administración.
Interfase gráfica – Gnome, KDE, etc.**
AUTOR: Jesús Garrido
Actualización 2015

ÍNDICE

1.	INTRODUCCIÓN	4
2.	EL SISTEMA OPERATIVO UNIX	5
2.1	HISTORIA DE UNIX.....	5
2.2	ESTANDARIZACIÓN DE UNIX.....	5
2.3	APARICION LINUX.....	6
2.4	CONCEPTOS GENERALES.....	6
2.4.1	Principios de diseño de UNIX.....	7
2.4.2	Principios de diseño de Linux.....	7
2.5	ARQUITECTURA DE UNIX - LINUX.....	7
2.6	MODOS DE FUNCIONAMIENTO DE UNIX.....	8
2.7	ELEMENTOS DEL SISTEMA UNIX/LINUX.....	8
2.7.1	Kernel.....	8
2.7.2	Procesos o Threads.....	10
2.7.3	Gestión de memoria.....	12
2.7.4	Sistema de Archivos.....	13
2.7.5	Gestión de Entrada y Salida.....	15
2.8	INTERRUPCIONES Y EXCEPCIONES.....	16
2.9	UTILIDADES O TOOLS.....	16
3.	ADMINISTRACIÓN UNIX - LINUX.....	17
3.1	RESPONSABILIDADES DEL ADMINISTRADOR.....	17
3.2	INSTALACIÓN.....	17
3.3	ARRANQUE DEL SISTEMA Y SERVICIOS.....	19
3.4	OBSERVACIÓN DEL SISTEMA.....	19
3.4.1	Logs de arranque del sistema.....	19
3.4.2	Kernel : Directorio /proc.....	20
3.4.3	Procesos.....	21
3.4.4	Trabajos no interactivos.....	21
3.4.5	Logs del sistema.....	21
3.4.6	Memoria.....	22
3.4.7	Discos y filesystems.....	22
3.4.8	Estructura del sistema de archivos de Linux.....	22
3.4.9	Servidores de impresión.....	24
3.5	ADMINISTRACIÓN DE SEGURIDAD.....	24
3.5.1	Usuarios y grupos.....	24
3.5.2	IDs de presentación y contraseñas.....	25
3.5.3	El superusuario (root).....	25
3.5.4	Fichero de contraseñas.....	25
3.5.5	Protección de datos frente a otros usuarios.....	26
3.5.6	Cifrado de ficheros.....	26
3.5.7	Seguridad en Linux.....	27
3.6	ADMINISTRACIÓN DE RED.....	31
3.6.1	NFS (Network File System).....	31
3.6.2	RFS (Remote File Sharing).....	31

3.7	HERRAMIENTAS INTEGRADAS DE ADMINISTRACIÓN	32
4.	INTERFAZ USUARIO	33
4.1	INTERFAZ MODO COMANDO: SHELL.....	33
4.1.1	El Shell Korn	34
4.1.2	El Shell CSH	34
4.1.3	El Shell Bash.....	35
4.2	INTERFACES GRÁFICOS.....	35
4.2.1	Arquitectura X - Windows	36
4.2.2	Gestor de ventanas (X Windows Manager)	37
4.2.3	Entorno de escritorio	37
4.3	KDE (K DESKTOP ENVIRONMENT) - PLASMA	39
4.3.1	Elementos del KDE.....	40
4.3.2	Panel.....	40
4.3.3	Panel de configuración	40
4.3.4	Ejecución de los programas en un icono	41
4.3.5	Herramientas del panel	41
4.3.6	Escritorios virtuales	41
4.3.7	Otros aspectos	41
4.4	GNOME	41
4.4.1	Características de GNOME	42
4.4.2	Gestor de escritorio.....	42
4.4.3	Panel GNOME	42
4.4.4	Otros aspectos de GNOME	42
	PREGUNTAS DE TEST.....	44
	SOLUCIONES A LAS PREGUNTAS DE TEST.....	47
	BIBLIOGRAFÍA BÁSICA.....	48

1. Introducción

Uno de los sistemas operativos más populares en los últimos años es UNIX. Su popularidad, además, se ha visto incrementado con la aparición de Linux como distribución bajo licencia de software libre.

Tanto UNIX como Linux son sistemas operativos que han evolucionado bastante a lo largo de su historia, originando distintas versiones, incompatibles, en algunas ocasiones, entre sí (caso de UNIX).

Uno de los aspectos más importantes en el mantenimiento de cualquier sistema operativo es su administración. Esta tarea es parte fundamental, ya que los errores producidos en esta faceta pueden ser difíciles de subsanar y originar inestabilidad en el funcionamiento del sistema operativo.

En las tareas de administración se exige un profundo conocimiento tanto en la forma de trabajar del sistema operativo como el hardware sobre el que está ejecutándose.

En la actualidad, casi todos los fabricantes proporcionan herramientas que facilitan la administración, aunque el principal problema es que son incompatibles de unos fabricantes a otros.

2. El sistema operativo UNIX

2.1 Historia de UNIX

Unix es uno de los sistemas operativos más ampliamente usados en computadoras que varían desde las personales hasta los mainframes. Existen diferentes versiones para máquinas uniprocador / multiprocador.

Unix a lo largo de su historia, evoluciona en los Laboratorios Bell de AT&T con un simulador de un viaje espacial en el sistema solar, su expansión se produce en las universidades y las versiones más importantes son creadas por la Universidad de Berkeley y AT&T con UNIX SYSTEM V

El sistema operativo UNIX fue creado por un equipo del Laboratorio Bell de la AT&T a principios de los 70: Ken Thompson y Dennis Ritchie se basaron en el proyecto MULTICS, que había quedado abandonado. Después de una primera versión en ensamblador PDP-7 denominada UNICS que posteriormente pasó a llamarse UNIX, se describe en un lenguaje de alto nivel, llamado **B** y posteriormente, Thompson y Ritchie desarrollan el **C** y lo vuelven a describir. En 1976 se difunden gratuitamente los fuentes de UNIX por las universidades de EUA constituyendo la Versión 6.

1978: Versión 7

1978-8x: Versión de Berkeley (BSD). Incorpora gestión de memoria virtual, utilidades, soporte de redes (TCP, sockets)

1983: Aparece el System V

198x: Formación de consorcios para fijación de estándares: XPG, OSF, etc. Normas POSIX. IBM y DEC empiezan a emplear UNIX

Finales de 198x: Surgen las versiones 4.3BSD y System V R3, que son incompatibles.

199x: Linux (UNIX gratuito)

2.2 Estandarización de UNIX

Debido a las múltiples versiones en el mercado de UNIX, se comenzaron a publicar estándares para que todas las versiones fuesen 'compatibles'. La primera de ellas la lanzó AT&T llamada SVID (System V Interface Definition) que definiría cómo deberían ser las llamadas al sistema, el formato de los archivos y muchas cosas más. La otra versión importante, la de Berkeley (Berkeley Software Distribution o BSD) simplemente ignoró el proyecto anterior. El primer intento serio de reconciliación se inició bajo los auspicios de la IEEE con un proyecto colectivo denominado **POSIX** de "Portable Operating System". Con la colaboración de personas de la industria, las universidades y el gobierno el comité POSIX produjo el estándar conocido como **1003.1** que define un **conjunto de procedimientos de biblioteca que debe proporcionar todo sistema UNIX que cumpla con la norma**. Posteriormente los institutos ANSI e ISO se interesaron en estandarizar el lenguaje 'C' y conjuntamente se publicaron definiciones estándares para otras áreas del sistema operativo como la interconectividad, el intérprete de comandos y otras. En la tabla 1 se muestran las definiciones de POSIX.

Estándar	Descripción
1003.0	Introducción y repaso.
1003.1	Procedimientos de biblioteca
1003.2	Intérprete y comandos.
1003.3	Métodos de prueba.
1003.4	Extensiones para tiempo real.
1003.5	Lenguaje Ada.
1003.6	Extensiones para la seguridad
1003.7	Administración del Sistema.
1003.8	Acceso transparente a archivos.
1003.9	Lenguaje Fortran.
1003.10	Supercómputo.

Tabla 1. Las Especificaciones de POSIX

Al momento del auge de los estándares de POSIX, se formó un grupo de fabricantes de computadoras (IBM, DEC y Hewlett-Packard) que lanzaron su propia versión de UNIX llamada **OSF/1** (de **Open Software Foundation**). Lo bueno fue que su versión tenía como objetivo cumplir con todos los estándares del IEEE, además de un sistema de ventanas (el **X11**), una interfaz amigable para los usuarios (**MOTIF**) y las definiciones para cómputo distribuido (**DCE**) y administración distribuida (**DME**). La idea de ofrecer una interfaz amigable en UNIX no fue original de OSF, ya en la versión 3.5 de SunOS de Sun Microsystems se ofrecía una interfaz amigable y un conjunto de librerías para crear aplicaciones con interfaz gráfica técnicamente eficiente y poderosa llamada SunWindows o SunVIEW. Esta interfaz junto con sus librerías estaba evolucionando desde la versión para máquinas aisladas hacia una versión en red, donde las aplicaciones podían estar ejecutando en un nodo de la red y los resultados gráficos verlos en otro nodo de la red, pero Sun tardó tanto en liberarlo que le dio tiempo al MIT de lanzar el X11 y ganarle en popularidad.

AT&T formó, junto con Sun Microsystems y otras compañías UNIX International y su versión de UNIX, provocando así el que ahora se manejen esas dos corrientes principales en UNIX.

2.3 Aparición LINUX

El sistema operativo Linux se genera inspirándose en otros dos sistemas operativos:

- El sistema abierto UNIX creado en 1969 por Ken Thompson y Dennis Ritchie en los laboratorios de Bell
- El sistema educativo Minix creado en 1987 por Andrew S. Tanenbaum.

Del primero de los sistemas se toman sus características, especificaciones y funcionamiento. Del segundo se adopta la estructura y código del núcleo. Con todo esto en 1991 Linus Torvalds crea Linus's Unix = Linux Kernel, esto es, crea solo el núcleo del sistema sin la capa de servidores, manejadores, aplicaciones gráficas, etc. que serán creadas posteriormente por otros autores. El código del núcleo lo podemos encontrar en la dirección (www.kernel.org). El núcleo actual tiene aproximadamente 1,5 millones de líneas de código, y representa menos del 50 por ciento de todo el código del sistema.

En la comunidad de programadores se crea el proyecto GNU (Gnu's Not Unix), proyecto para generar software libre, donde se generan editores Emacs, compiladores gcc, interprete de comandos bsh, sistema operativo Hurd, aplicaciones, etc., bajo la licencia pública general GPL (General Public License), usar, copiar, distribuir y modificar (con las mismas condiciones). Se conserva la firma del autor. Se puede cobrar.

Linux se crea con esta filosofía de libre distribución y el sistema operativo completo que se construye con este núcleo también. A todo el sistema se le da el nombre de GNU/Linux (distribución completa del sistema operativo con Linux), que contiene el núcleo (1,5 millones de líneas de código) mas las otras capas del sistema operativo y utilidades. Si bien muchas veces se denomina a todo el sistema simplemente LINUX.

2.4 Conceptos generales

UNIX es un sistema que, en sus inicios, fue diseñado por y para programadores. Ofrece grandes facilidades a la programación: compiladores, enlazadores, entornos de desarrollo, etc.

La 'filosofía' de UNIX es la escribir programas que hagan una sola cosa, y que la hagan bien: Es por tanto simple y modular. Escribe programas que trabajen bien juntos: En **tubas** o Pipes. Escribe programas que manejen tramas de texto, porque es la interfaz universal: La consecuencia es la de manejar los dispositivos como ficheros simples

Las claves técnicas del éxito de UNIX son:

- Es simple y coherente
- Permite aprovechar el trabajo de los demás
- Es muy flexible: desde el punto de vista del hardware, del tamaño, de las aplicaciones, etc.

Se puede decir que Linux en realidad es GNU/Linux aunque en el futuro puede llegar a ser GNU/Hurd. Tiene un peculiar sistema de desarrollo: Hay miles de programadores, muy organizados y sometidos a una férrea disciplina, que de forma voluntaria y en su tiempo libre realizan los desarrollos.

2.4.1 Principios de diseño de UNIX

Los principios fundamentales de Unix y algunas de las razones de su éxito son:

- **Interactivo:** Acepta órdenes las ejecutar y espera más órdenes
- **Multiusuario:** Puede ser usado por más de una persona simultáneamente
- **Multitarea:** Puede realizar a la vez varios trabajos (procesos)
- Razonablemente **seguro**. Existe protección de acceso a los recursos
- **Multiplataforma:** Se potencia la portabilidad de las aplicaciones a nivel de código fuente. Soporta múltiples arquitecturas de procesadores de 16, 32 y 64 bits (Intel x86, Sparc, PowerPC, Alpha, etc.)

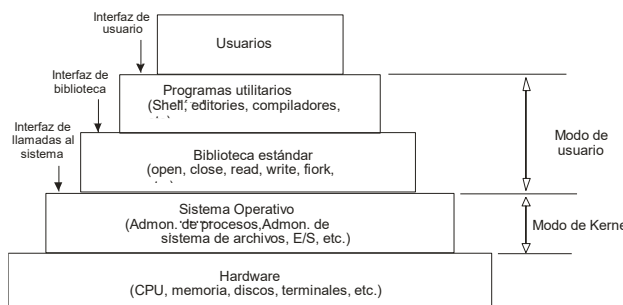
2.4.2 Principios de diseño de Linux

En cuanto a Linux podemos decir:

- Sus principios de diseño son similares a los de UNIX. Se le puede considerar como un **clon UNIX**
- Es **'Free'** como libre: Software libre bajo la licencia GPL
- Ideal para programadores: Proporciona gran variedad de compiladores, entornos de desarrollo, librerías, escritorios, gestores de ventanas, etc.
- Está muy enfocado a comunicaciones y redes, incluyendo también acceso a **Internet**
- **Es multiplataforma en cuanto al Hardware:** Se adapta a todo tipo de máquinas, desde las muy pequeñas (ej. Teléfonos móviles) a las más grandes (mainframe)
- **Es un sistema operativo de propósito general.** Desde oficina a cómputo distribuido, 3DFX, etc. Está basado en miles de paquetes de software libre, agrupados mediante **distribuciones**

2.5 Arquitectura de UNIX - Linux

La arquitectura de Unix está basada en niveles tal y como se muestra en la figura. El traspaso de información de un nivel a otro se hace a través de *cepos o traps*



En la capa inferior tenemos toda la parte del Hardware que el sistema operativo debe gestionar. Por encima de éste se sitúa el *kernel* de Unix que es el encargado de la administración de procesos, gestión del sistema de archivos, entradas / salidas, etc. A los procesos que trabajan a ese nivel se les llama procesos en *modo kernel*.

La biblioteca estándar se ubica por encima del *kernel*, se encarga, por ejemplo de las operaciones de *open*, *close*, etc. A este nivel se trabaja en modo usuario. La interface entre las dos capas, o el acceso de la capa de biblioteca estándar a la del kernel se realiza a través de la **interfaz de llamadas al sistema**.

A un nivel superior tenemos los programas y utilidades como el shell, compiladores, etc., que sirve de ayuda a desarrolladores y usuarios que interactúan con el sistema operativo. La interface entre esta capa la inmediatamente inferior es a través de la **interfaz de biblioteca**.

Por último, se ubicarían los usuarios que por medio de la interfaz de usuario se comunican con el shell, u otras utilidades del sistema Unix.

2.6 Modos de funcionamiento de UNIX

Los dos modos de funcionamiento de los procesos, correspondientes a sus niveles de ejecución, son: el modo usuario y el modo kernel. Ambos modos son autónomos pero no independientes. El kernel es parte de cada proceso de usuario

El paso de modo usuario a kernel o viceversa se realiza a través de cepos a traps que crean una interrupción para acceder a la interfase de llamadas al sistema y al resto de los componentes de nivel kernel. El paso del modo kernel a usuario es un retorno tras la realización de la petición que motivó el paso al modo kernel.

2.7 Elementos del sistema Unix/Linux

Los componentes del sistema Unix - Linux son:

- Kernel
- Procesos y threads
- Gestión de memoria
- Entrada / salida
- El sistema de archivos

2.7.1 Kernel

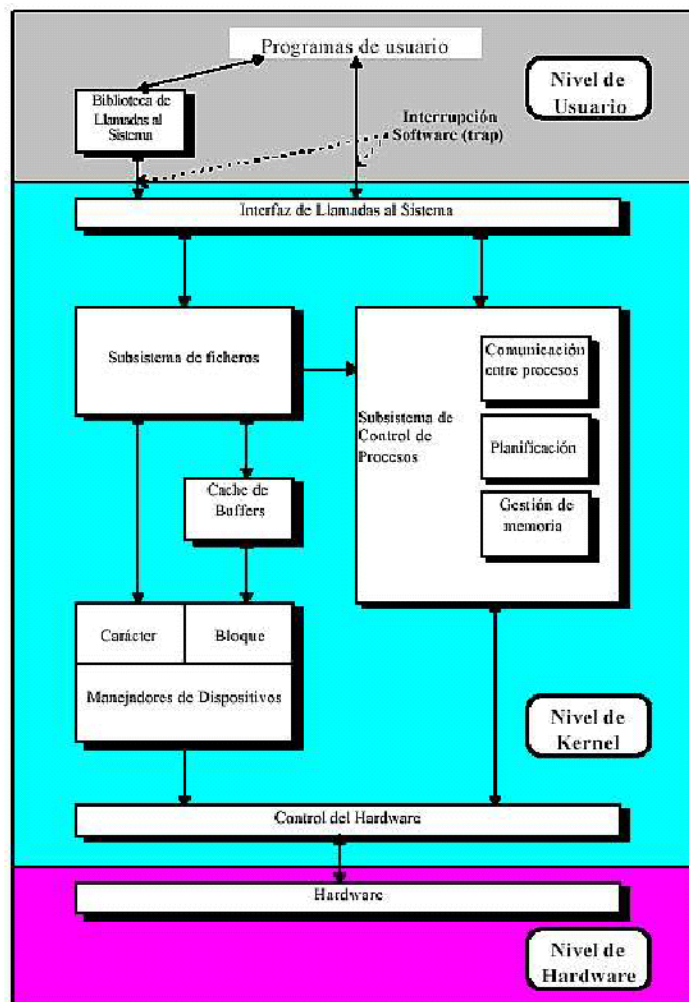
El *Kernel* es el corazón de Unix. Se clasifica como de tipo monolítico pero permite la incorporación dinámica de módulos. En él se pueden encontrar dos partes principales: El núcleo dependiente y el núcleo independiente

Núcleo dependiente: Se encarga de las interrupciones, los manejadores de dispositivos de bajo nivel (lower half) y parte del manejo de la memoria.

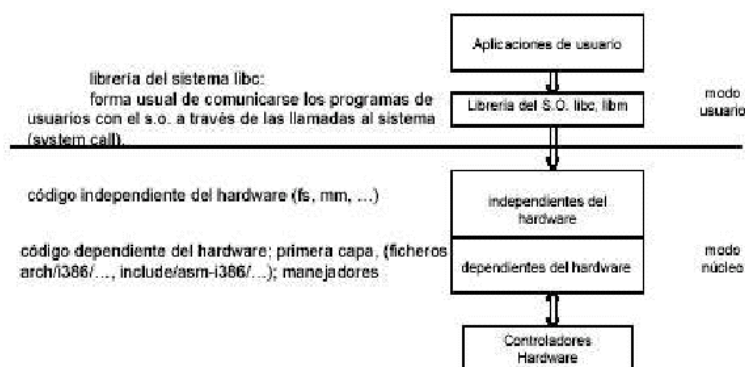
Núcleo independiente: Es igual en todas las plataformas e incluye:

- Manejo de llamadas del sistema
- Controlar la ejecución de procesos, permitiendo su creación, terminación o suspensión y comunicación
- Planificar los procesos para su realización por la CPU. Los procesos la comparten mediante un sistema de tiempo compartido
- Asignar memoria principal a un proceso en ejecución. Permite que los procesos compartan porciones de su espacio de direcciones bajo ciertas condiciones, pero impide que un proceso tenga un espacio específico de direcciones.
- Asigna memoria secundaria para lograr un almacenamiento y recuperación eficiente de los datos de usuario. Asigna almacenamiento secundario para los archivos de usuario, recupera el almacenamiento no empleado, estructura el sistema de archivos de un modo compresible y los protege del acceso de usuario no autorizados
- Permite el acceso controlado de los procesos a los periféricos tales como: terminales, unidades de disco, equipos de red.

El esquema de Unix/Linux sería el que marca la siguiente figura: Los usuarios o sus procesos a través de traps y la interfaz de llamadas al sistema acceden al modo kernel del sistema operativo. Los principales módulos a los que accederán serán el subsistema de control de procesos (gestión de memoria, planificación, comunicación entre procesos) y al subsistema de ficheros y manejadores de dispositivos.



Entrando un poco más en detalle (véase la figura siguiente), para la comunicación entre el modo *usuario* y el modo *kernel*, se observa que Unix/Linux proporciona unas librerías: *libc* y *libm* que son llamadas por los procesos de usuario cuando cambian de modo (pasan del modo usuario al modo kernel, por ejemplo al solicitar una operación de entrada / salida) a través de las *llamadas al sistema* o *system calls*. Posteriormente se accede al *núcleo independiente* (escrito en código independiente), para acceder al *núcleo dependiente*, cuyo código depende de la plataforma hardware



Otra característica del Kernel de Unix/Linux es que puede soportar varias CPU's (SMP, Symmetric MultiProcessing). También soporta procesos en tiempo real (Real Time según POSIX 1003.1b). Es paramétrico configurable mediante numerosos de parámetros y se carga en memoria de forma residente en memoria al arrancar el sistema.

2.7.2 Procesos o Threads

Un programa es un fichero ejecutable y un proceso es una instancia de un programa en Ejecución. Esto quiere decir que, si se arranca tres veces un mismo programa, observaremos que en memoria aparecerá ese programa tres veces.

Un proceso es un conjunto de bytes que la CPU interpreta como intrusiones HW (elementos texto) datos y pila. Como complemento al texto está el contexto de un proceso, que no es sino su estado definido por su texto; los valores de sus variables de usuario y estructuras de datos globales; los valores de los registros que la máquina emplea; los valores almacenados en su ranura de la tabla de procesos y en la user area y; los contenidos de sus pilas de usuario y kernel (una por cada modo). Cuando el ordenador pasa de ejecutar un proceso A otro B, hace un cambio de contexto, el ordenador deja de trabajar en el contexto A y pasar a trabajar en el contexto B.

El cambio de contexto exige, en caso de no haber terminado el proceso A, salvar todo el estado del ordenador según corresponde a este proceso para restituir o iniciar el estado del proceso B. Este estado no se limita a los registros generales, contador de programa y al registro de estados de la máquina, esto es al estado visible al programador, sino que engloba el estado de todos los recursos físico y lógicos utilizados por el proceso. Así desde el punto de vista de los recursos físicos, es necesario salvar todos los registros internos del ordenador que son necesario para mantener la protección de memoria, para realizar la traducción de direcciones, para establecer el nivel de interrupciones, etc. Desde el punto de vista de los recursos lógicos, es necesario almacenar el estado de los ficheros abiertos por el proceso.

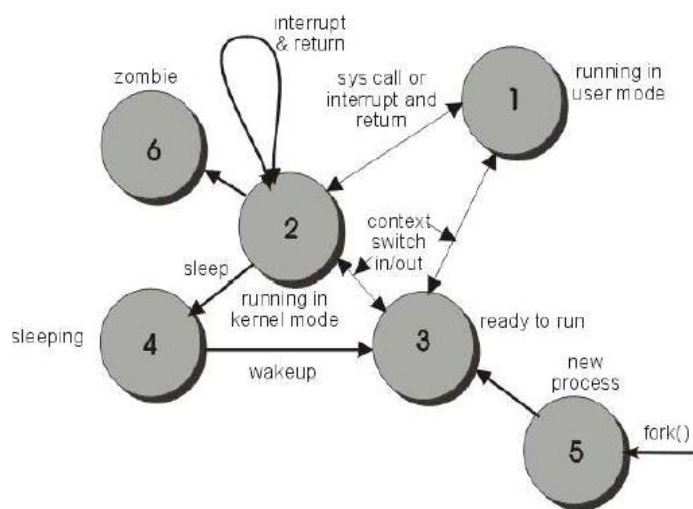
El número total de estados es 9:

1. Ejecución en modo usuario
2. Ejecución en modo kernel
3. El proceso no está en ejecución, está *ready to run*, pero se halla preparado para funcionar a la espera de que el kernel lo decida,
4. El proceso no está en ejecución, pero está listo a la espera de que el swapper lo transfiera a memoria principal para que el kernel decida su ejecución

CONTENIDO

5. El proceso está durmiendo (sleeping) en la memoria principal
6. El proceso está sleeping y el swapper lo ha transferido al almacenamiento secundario para hacer sitio en la memoria principal
7. El proceso está volviendo desde el kernel al modo usuario, pero el kernel le da preeminencia a este último y hace un cambio de contexto para realizar otro proceso
8. El proceso acaba de ser creado y se encuentra en estado de transición, existe pero no se halla preparado para funcionar ni está durmiendo
9. El proceso ha realizado la llamada exit y se encuentra en estado fantasma, ya no existe pero deja un registro con un código de salida y alguna información para el proceso padre

La siguiente figura es un esquema de transición entre algunos estados que sufre un proceso



El **subsistema de control de procesos (Process Control Subsystem)** es el responsable de:

- Comunicación entre procesos
- Gestión de memoria
- Actuación del planificador

El **módulo de comunicación entre procesos** controla todas las posibilidades, desde señalización asíncrona de sucesos hasta transmisión síncrona de mensaje entre procesos

El **scheduler (planificador)** asigna CPU a los procesos. Estos se realizan hasta que la abandonan en espera de un recurso solicitado o hasta que el kernel hace valer su función cuando el tiempo de funcionamiento más próximo excede un valor prefijado. Posteriormente, el scheduler escogerá el proceso con mayor prioridad

El manejo de procesos en UNIX es por **prioridad** y **round robin**. En algunas versiones se maneja también un **ajuste dinámico de la prioridad** de acuerdo con el tiempo que los procesos han esperado y al tiempo que ya han usado el CPU. El sistema provee facilidades para contabilizar el uso de CPU por proceso y una pila común para todos los procesos cuando necesitan estar ejecutando en modo privilegiado (cuando hicieron una llamada al sistema).

UNIX permite que un proceso haga una **copia de sí mismo** por medio de la llamada **'fork'**, lo cual es muy útil cuando se realizan trabajos paralelos o concurrentes; también se proveen facilidades para el envío de **mensajes entre procesos (pipes, signals)**.

Los procesos no interactivos se denominan **daemons** o procesos background. Cuando se inicia un proceso, se le asigna un identificador **PID**, se guarda el proceso que lo lanzó **PPID**, el propietario que lo lanzó **UID** y el grupo de pertenencia **GID**, lo que definirá el perfil de permisos de acceso a los que tendrá derecho. Existe la posibilidad de alterar el usuario o grupo efectivo de permisos, durante la ejecución del proceso, mediante la llamada a **setuid** o **setgid**, siempre que se disponga de los permisos apropiados. También existe una bandera de permisos **setuid** asociada al archivo del programa que permite ejecutar éste, con los permisos del propietario del archivo en lugar de los del usuario que lo ejecuta.

Linux combina multiprogramación y tiempo compartido. Es un sistema preemptive, con planificación dependiente de la categoría del proceso: tiempo real o proceso ordinario. Sobre procesos ordinarios emplea la técnica de planificación por prioridad dinámica (la asignada inicialmente más una variable asignada por el sistema). En tiempo real se aplica un planificador FIFO non-preemptive para ciertas tareas de igual prioridad o Round Robin preemptive en otros casos, también de tiempo real.

2.7.3 Gestión de memoria

Los sistemas UNIX utilizan el manejo de memoria virtual siendo el esquema más usado la **paginación por demanda** y **combinación de segmentos paginados**, en ambos casos con páginas de tamaño fijo. En todos los sistemas UNIX **se usa una partición de disco duro para el área de intercambio**. Esa área **se reserva al tiempo de instalación** del sistema operativo. Una regla muy difundida entre administradores de sistemas es asignar una partición de disco duro que sea al menos el doble de la cantidad de memoria real del ordenador. Con esta regla se permite que se puedan intercambiar flexiblemente todos los procesos que estén en memoria RAM en un momento dado por otros que estén en el disco. Todos **los procesos que forman parte del kernel no pueden ser intercambiados** a disco.

Algunos sistemas operativos (como SunOS) permiten incrementar el espacio de intercambio incluso mientras el sistema está en uso (en el caso de SunOS con el comando 'swapon'). También es muy importante que al momento de decidirse por un sistema operativo se pregunte por esa facilidad de incrementar el espacio de intercambio, así como la facilidad de añadir módulos de memoria RAM a la computadora sin necesidad de reconfigurar el núcleo.



Cada proceso dispone de su propio espacio de direcciones, organizado en segmentos según:

Text Segment, que almacena el código

Data Segment almacena los datos o variables que utilizan los procesos, éste segmento tiene dos partes: **Initialized Data** -datos inicializados- y **Uninitialized Data** -datos no inicializados- conocido como **BSS**

Stack Segment: almacena la información referente a llamadas a otras funciones

Es posible compartir código entre procesos mediante el empleo de **Shared Text Segments**. Dos procesos nunca comparten los segmentos de datos y de pila (salvo los thread), la forma de compartir información se lleva a cabo mediante el empleo de segmentos especiales de memoria compartida **Shared Segments**.

Cuando se arranca el sistema operativo Unix/Linux, inicialmente se inician los siguientes procesos con sus correspondientes identificadores: Swapper (PID 0); Init (PID 1); Page Daemon (PID 2)

En 4BSD, la memoria principal consta de 3 partes. Las dos primeras, el *kernel* y el *mapa central* (**core MAP**), están fijas en la memoria y nunca se pagan a disco. El resto de la memoria se divide en marcos de página, cada uno de los cuales puede contener una página de texto (código), datos o de pila, contener una página de la tabla de páginas o estar en la lista libre.

El **mapa central** incluye información acerca del contenido de los marcos de página, donde cada entrada describe el marco de página correspondiente (la 0 el marco 0 y así consecutivamente).

Reemplazo: cada 250ms el page daemon mira si el número de marcos libres no baja de cierta cantidad, (aprox. ¼ del tamaño de la memoria). Para la paginación se emplea un algoritmo conocido como **algoritmo de reloj** de una manecilla, examinando de forma circular los marcos de página. Es un algoritmo global. 4BSD emplea una versión modificada con 2 "manecillas", mientras que System V emplea el algoritmo de una manecilla, colocando los marcos en la lista de marcos libres, si no se usa durante *n* pasadas consecutivas. Si hay reemplazo el swapper desaloja uno o varios procesos que hayan estado inactivos durante más de 20s, o el que haya estado más tiempo en memoria entre los cuatro más grandes.

2.7.4 Sistema de Archivos

En UNIX, el sistema de archivos tiene una estructura de capas. Un fichero es una secuencia de 0 o más bytes. No existe distinción entre ficheros ASCII, binarios, etc. Se admiten nombres de hasta 255 caracteres. **No** siguen el esquema habitual de *nombre.extensión*

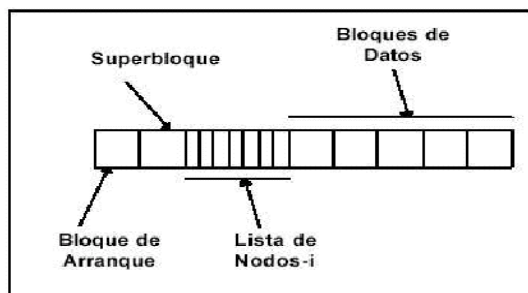
Existen varias versiones del sistema de archivos UNIX:

- Virtual File System (VFS) de SVR4
- Fast Files System (FFS) de Berkeley
- Network File System (NFS) de Sun
- Remote File System (RFS) de ATT

VFS (Virtual File System)			
UFS	NFS	RaiserFS	CDFS
Driver Volúmenes Lógicos			
Drivers de Disco			
Disco 1	Disco 2	Dispositivo Raid 1	Dispositivo Raid 2

Cada partición de un disco tiene una estructura:

- Bloque Boot: bloque de arranque
- Superbloque (nº de bloques, nº de i-nodes, comienzo de la lista de bloques libres)
- i-nodes: Es la representación interna de un fichero. Su contenido son punteros a los bloques que forman los datos del fichero y propiedades del fichero (UID propietario, GUID, permisos, fecha, etc.). Hay uno por fichero (el número de ellos está limitado en la creación del sistema de archivos). El i-nodo **NO contiene el nombre** del fichero. Un directorio mantiene una lista con los nombres de los ficheros y asocia a cada nombre de fichero su i-nodo correspondiente. Distintos nombres de fichero pueden estar asociados al mismo i-nodo.
- Bloques de datos.



Un fichero puede ocupar varios bloques no necesariamente contiguos. Un disco puede tener varias particiones con su propia estructura.

El sistema de archivos de UNIX, desde el punto de vista del usuario, tiene una organización jerárquica o de árbol invertido que parte de una raíz conocida como "/" (diagonal). Es una diagonal al revés que la usada en DOS. Se puede acceder a un fichero empleando un direccionamiento absoluto, desde el directorio raíz; relativo, desde el directorio actual; usando Enlaces (links), ubicaciones ficticias de ficheros que se encuentran en otro sitio.

Internamente se usa un sistema de direccionamiento de archivos de varios niveles, cuya estructura más primitiva se le llama 'information node' (i-node). El sistema de archivos de UNIX ofrece un poderoso conjunto de comandos y llamadas al sistema. En la tabla 3 se muestran los comandos más útiles para el manejo de archivos en UNIX.

UNIX	Utilidad
rm	Borra archivos
cp	Copia archivos
mv	Renombra archivos
ls	Lista el contenido de un directorio
mkdir	Crea un directorio
rmdir	Borra directorios
ln	Crea enlaces (entradas a un mismo archivo físico desde ubicaciones diferentes), tanto enlaces "físicos" (hard-links) como "simbólicos" (soft-links). Un enlace simbólico guarda la ruta del archivo referenciado, puede establecerse entre particiones distintas y no desaparece si se elimina el archivo a que hace referencia. Un enlace físico solamente se puede establecer en directorios de la misma partición, para eliminar un archivo se deben eliminar todos los enlaces físicos que lo referencien. Todos los enlaces físicos de un archivo comparten el propietario y los permisos de acceso.

Tabla 2 Comandos UNIX del Sistema de Archivos

Comentado [I1]: Manuel: no hay tabla 2, hay que cambiar este a tabla 2?

Dentro de la estructura de directorios de UNIX existen una serie de directorios comunes a todas las instalaciones que es preciso conocer:

- **/:** Directorio raíz, inicio del sistema de archivos.
- **/swap:** Directorio no accesible por el usuario, que hace referencia al área de swap del sistema.
- **/tmp:** Directorio de archivos temporales.
- **/dev:** Directorio de dispositivos. En él se encuentran todos los dispositivos de E/S, que se tratan como archivos especiales.
- **/etc:** Directorio para archivos de sistema diversos
- **/bin:** Directorio para programas binarios (ejecutables)

- **/lib:** Directorio de bibliotecas del sistema
- **/usr:** Directorio de usuarios
- **/home:** Directorio base a partir del cual se ubican los directorios por defecto de las cuentas de usuario.

Además del sistema de archivos clásico de UNIX, existen otros sistemas de archivos que pueden emplearse en UNIX y LINUX. Berkeley mejoró el sistema clásico con el **BFFS** (*Berkeley Fast File System*), que fue quien introdujo los nombres de archivo de 255 caracteres (el clásico solamente admitía 14). Berkeley tuvo que introducir cuatro llamadas al sistema nuevas **opendir**, **closedir**, **readdir** y **rewinddir** para que los programas pudieran leer los directorios sin conocer su estructura interna. Posteriormente estos cambios se añadieron a POSIX y a todas las versiones de UNIX.

LINUX comenzó empleando el sistema de archivos de MINIX, pero estaba limitado a nombres de 14 caracteres y a archivos de 64 MB de tamaño. La primera mejora vino de la mano de un sistema de archivos denominado **Ext** que permitía nombres de archivo de 255 caracteres y 2 GB de tamaño por archivo, pero era muy lento. La evolución vino de la mano del sistema de archivos **Ext2**, con nombres de archivo largos, archivos grandes y mejor rendimiento, siendo en la actualidad el sistema de archivos por defecto en LINUX. **Ext2** es muy similar a BFFS, con algunas diferencias menores.

Con objeto de compartir los archivos entre ordenadores conectados en una red, a menudo con diferentes sistemas de archivos, Sun Microsystems desarrolló el sistema de archivos de red **NFS** (*Network File System*), que se usa en todos los sistemas UNIX modernos, incluido LINUX.

El sistema de archivos es el esquema que organiza y gestiona los datos. Incluye un conjunto de programas y herramientas organizadas en una estructura jerárquica. Esta estructura permite a los usuarios acceder y actuar sobre los archivos con absoluta independencia de su localización física.

Cualquier sistema de archivos antes citado se caracteriza por:

- Una estructura jerárquica
- Tratamiento coherente de los archivos de datos
- Protección de estos archivos
- Capacidad de crear y destruir archivos
- Crecimiento dinámico de estos archivos
- Tratamiento de cualquier dispositivo periférico como archivo

2.7.5 Gestión de Entrada y Salida

Los dispositivos de entrada y salida son considerados ficheros especiales: Toda entrada / salida está basada en el principio de que todos los dispositivos se pueden tratar como ficheros simples. El subsistema de entrada / salida utiliza como elementos principales: Buffercache; el código general de manejo de dispositivos; y drivers de dispositivos de hardware. Existen dos tipos de dispositivos:

Dispositivos de bloques:

- Usan secuencias de bytes (bloques)
- Utilizan buffer-cache
- Están estructurados en bloques de tamaño fijo (512 bytes)
- Permiten optimizar el rendimiento

Dispositivos de carácter:

- Son dispositivos sin estructura, son **raw**
- No usan buffer
- Las operaciones se realizan carácter a carácter

2.8 Interrupciones y excepciones

UNIX permite a dispositivos como periféricos de I/O o al reloj del sistema interrumpir la CPU asíncronamente. Al recibir la interrupción, el kernel almacena el contexto actual, determina la causa y responde a la interrupción. Tras responder a ésta, devuelve el contexto interrumpido y sigue ejecutando. El HW asigna las prioridades a los dispositivos de acuerdo con el orden de actuación en las interrupciones.

Así como las interrupciones están causadas por factores externos a un proceso, las excepciones son sucesos inesperados producidos por procesos, tales como la ejecución de instrucciones reservadas, de forma que el sistema, al encontrarse con una, tiende a reiniciar la instrucción, en lugar de pasar a la siguiente.

No obstante, el kernel debe tener la posibilidad de impedir la aparición de interrupciones en momentos críticos, para evitar la degradación de los datos. El sistema que se utiliza es el de disponer de un conjunto de instrucciones restringidas que colocan el nivel de ejecución del procesador en el estado de palabra (status word). Al asignar un nivel de ejecución del procesado, todas las interrupciones de ese nivel e inferiores quedan suprimidas, permitiendo sólo las superiores.

Los 6 niveles típicos de interrupción son:

- Interrupciones por SW
- Terminales
- Dispositivos de red
- Disco
- Reloj
- Errores de máquina

2.9 Utilidades o tools

Las utilidades o tools son los programas estándar que utiliza UNIX para realizar las tareas solicitadas por un usuario o una aplicación.

3. Administración UNIX - Linux

3.1 Responsabilidades del administrador

La Administración del sistema es el mantenimiento rutinario necesario para que un sistema crezca y se modifique. El ingreso y la retirada de los identificadores de presentación del usuario, la instalación de nuevo software, la eliminación de ficheros de registro, la realización de copias de seguridad, y la instalación de parches de sistema operativo y actualización de paquetes software para corrección de bugs funcionales o vulnerabilidades de seguridad son tareas que caen dentro del ámbito de administración del sistema. Un usuario de una máquina UNIX está generalmente designado como administrador del sistema, y es tarea del administrador mantener el sistema activo y en ejecución al servicio de otros usuarios

El administrador del sistema es generalmente un usuario individual, responsable de mantener el sistema en ejecución correctamente. UNIX proporciona un id de presentación especial llamado root. El id root tiene su propia contraseña y también privilegios especiales, incluyendo acceso completo a todos los ficheros y recursos del sistema. La presentación root se denomina superusuario y siempre debe tener una contraseña.

Otras responsabilidades del administrador del sistema son:

1. Apagar y arrancar el sistema. Existen diferentes niveles de ejecución:

- Nivel 1: modo halt y apagado
- Nivel 2: modo monousuario
- Nivel 3: modo multiusuario

En Linux se pueden añadir más niveles:

- Nivel 4: modo multiusuario y red
- Nivel 5: entorno X

2. Administración de seguridad: Control de acceso de los usuarios. Gestión de usuario y sus permisos. Implica la creación y gestión de las cuentas de usuario del sistema y de gestionar los permisos que se asignan a cada usuario

3. Gestión del espacio físico: Gestionar las particiones y unidades de disco y backup que el sistema monta automáticamente en su arranque

4. Mantenimiento de archivos: Backups, chequeo de los discos, etc.

5. Monitorización del sistema: Comprobaciones del uso de CPU, discos, memoria, gestión de entrada / salida, red. Mediciones de throughput

6. Identificación de las causas de los problemas y su resolución.

3.2 Instalación

Al contrario que lo que ocurre con **Microsoft Windows** la instalación de **Linux** no es un proceso sencillo, puesto que **Linux** permite el control y la personalización de una cantidad mayor de parámetros y opciones. Pese a todo se están realizando grandes progresos buscando que la instalación de **Linux** sea un proceso lo menos traumático posible, dependiendo la sencillez de la

misma de la distribución que se emplee. Por el momento la distribución más sencilla de instalar es **Ubuntu** hecho que la ha hecho muy popular. Ahora bien, la evolución del resto de distribuciones también ha simplificado mucho su instalación.

Pese a todo antes de proceder a instalar **Linux** es necesario tener en cuenta una serie de aspectos fundamentales. El primero de ellos es leer la información que contiene el **iso o cd** de la instalación, esta información puede aparecer de dos formas distintas, los llamados **HOWTO** o en forma de manuales desarrollados para la distribución. Las distribuciones incluyen un manual de instalación de **Linux** en **HTML o pdf** en la que se explica con todo lujo de detalles cómo se realiza ésta y que es conveniente leer y comprender antes de proceder a realizar la instalación.

Uno de los conceptos principales para tener en cuenta antes de la instalación es el de **partición**. Cada sistema operativo organiza la información de los ficheros que contiene de forma diferente, utilizando cada uno su propio sistema de archivos.

Esto en general impide que se puedan instalar varios sistemas operativos **mezclados** en un mismo disco duro. Para solucionar este problema existen las llamadas **particiones** con las que se divide un determinado disco duro de forma que pueda contener ambos sistemas de archivos. A todos los efectos realizar una partición es equivalente a que el disco duro se divida en dos.

Los problemas fundamentales al instalar **Linux** provienen de que en la mayor parte de las ocasiones el usuario desea conservar **Windows** y todos los programas para este sistema. En la actualidad existen varias distribuciones que permiten la instalación de **Linux** en un sistema de archivos de **Windows**, bien en lo que se denomina un disco imagen (un fichero muy grande). No obstante, para obtener un buen rendimiento es preferible instalar **Linux** sobre una partición diferente de la de **Windows** empleando el sistema de archivos propio de **Linux**, por lo que suele ser necesario realizar una partición del disco duro (Habitualmente **Windows** se apropia de la totalidad). Hasta épocas recientes esta división suponía la pérdida irremediable e inevitable de toda la información que contuviese el disco duro. En la actualidad las distribuciones de **Linux** incluyen utilidades como **fips** que permite dividir el disco duro sin perder información.

Linux necesita al menos dos particiones para operar. Estas particiones deben crearse al momento de la instalación. Las dos particiones necesarias se deben asignar para el sistema de archivos en / y el área de intercambio o swap.

Partición Montaje Formato Tamaño

/dev/sda1 – Swap – swap – 10 Gb

/dev/sda2 – / – Ext4 – 240 Gb

Estas dos particiones se crean automáticamente en la mayoría de las distribuciones actuales dedicadas al escritorio y es la forma general y más sencilla de distribuir el sistema de archivos y las particiones. Sin embargo, en un servidor las particiones toman mayor importancia ya que en estos sistemas no solo se busca que funcione, sino que además se un sistema robusto, que el sistema siga funcionando aun cuando se acabe el espacio, etc. Y en estos casos las particiones juegan un papel importante.

Otro de los puntos para tener en cuenta es cómo se quiere arrancar **Linux** si existe **Windows**, la forma más sencilla es instalar **LILO o GRUB**, un programa que se encarga de arrancar ambos sistemas operativos según lo que indique el usuario al arrancar el PC.

Por último existe otro concepto fundamental a la hora de instalar y usar **Linux** que es el de **Super Usuario** o usuario **root**. Este usuario es el administrador del sistema y se crea durante la instalación. Como administrador que es, puede acceder y modificar (así como destruir) toda la información del sistema, por lo que hay que evitar en la medida de lo posible trabajar como usuario **root**.

3.3 Arranque del sistema y servicios

Una de las primeras tareas con la que tendrá que enfrentarse el administrador será la gestión de los recursos locales presentes en la máquina.

Comenzaremos por analizar el proceso de arranque de un sistema GNU/Linux, que nos hará comprender la estructura inicial del sistema y su relación con los diversos servicios que éste proporciona.

Un primer punto importante en el análisis del comportamiento local del sistema es su posible funcionamiento en los llamados niveles de ejecución (o *runlevels*), y en los servicios que se proporcionan en cada nivel.

Un servicio es una funcionalidad proporcionada por la máquina. La activación o parada de servicios se realiza mediante la utilización de *scripts*. La mayoría de los servicios estándar, los cuales suelen tener su configuración en el directorio */etc*, suelen controlarse mediante los *scripts* presentes en */etc/init.d/*. En este directorio suelen aparecer *scripts* con nombres similares al servicio donde van destinados, y se suelen aceptar parámetros de activación o parada.

- */etc/init.d/servicio start* arranque del servicio.
- */etc/init.d/servicio stop* parada del servicio.

Cuando un sistema GNU/Linux arranca, primero se carga el *kernel* del sistema, después se inicia el primer proceso, denominado *init*, que es el responsable de ejecutar y activar el resto del sistema, mediante la gestión de los niveles de ejecución (o *runlevels*).

Los niveles típicos son:

Runlevel	Función	Descripción
0	Parada	Finaliza servicios y programas activos, así como desmonta <i>filesystems</i> activos y para la CPU.
1	Monousuario	Finaliza la mayoría de servicios, permitiendo sólo la entrada del administrador (<i>root</i>). Se usa para tareas de mantenimiento y corrección de errores críticos.
2	Multiusuario sin red	No se inician servicios de red, permitiendo sólo entradas locales en el sistema.
3	Multiusuario	Inicia todos los servicios excepto los gráficos asociados a X Window.
4	Multiusuario	No suele usarse, típicamente es igual que el 3.
5	Multiusuario	Igual que 3, pero con soporte X para la entrada de usuarios (<i>login</i> gráfico).
6	Reinicio	Para todos los programas y servicios. Reinicia el sistema.

Después, el sistema, según el *runlevel* escogido, consulta los ficheros contenidos en */etc/rcn.d*, donde *n* es el número asociado al *runlevel*, en el que se encuentra una lista de servicios por activar o parar

3.4 Observación del Sistema

Una de las principales tareas del administrador (*root*) en su día a día, será verificar el correcto funcionamiento del sistema y vigilar la existencia de posibles errores o de saturación de los recursos de la máquina (memoria, discos, etc.). Pasaremos a detallar en los siguientes subapartados los métodos básicos para examinar el estado del sistema en un determinado momento y llevar a cabo las acciones necesarias para evitar problemas posteriores.

3.4.1 Logs de arranque del sistema

En el arranque de un sistema GNU/Linux, se produce todo un volcado de información interesante; cuando el sistema arranca, suelen aparecer los datos de detección de las características de la máquina, detección de dispositivos, arranque de servicios de sistema, etc., y se mencionan los problemas aparecidos.

Necesitaremos una serie de herramientas para este proceso.

- Comando *dmesg*: da los mensajes del último arranque del *kernel*.

- Fichero `/var/log/messages`: *log* general del sistema, que contiene los mensajes generados por el *kernel* y otros *daemons*.
- Comando `uptime`: indica cuánto tiempo hace que el sistema está activo.
- Ficheros `/proc`: ficheros que utiliza el *kernel* para almacenar la información que gestiona

3.4.2 Kernel: Directorio `/proc`

El *kernel* durante su arranque pone en funcionamiento un *pseudo-filesystem* llamado `/proc`, donde vuelca la información que recopila de la máquina, así como muchos de sus datos internos. El directorio `/proc` está implementado sobre memoria y no se guarda en disco. Los datos contenidos son tanto de naturaleza estática como dinámica (varían durante la ejecución).

Una de las características interesantes es que en el directorio `/proc` podremos encontrar las imágenes de los procesos en ejecución, junto con la información que el *kernel* maneja acerca de ellos. Cada proceso del sistema se puede encontrar en el directorio `/proc/pid-proceso`, donde hay un directorio con ficheros que representan su estado. Esta información es útil para programas de depuración, o bien para los propios comandos del sistema como `ps` o `top`, que pueden utilizarla para ver el estado de los procesos.

Por otra parte, en `/proc` podemos encontrar otros ficheros de estado global del sistema. Comentamos brevemente algunos ficheros que podremos examinar:

Fichero	Descripción
<code>/proc/bus</code>	Directorio con información de los buses PCI y USB
<code>/proc/cmdline</code>	Línea de arranque del <i>kernel</i>
<code>/proc/cpuinfo</code>	Información de la CPU
<code>/proc/devices</code>	Dispositivos del sistema de caracteres o bloques
<code>/proc/driver</code>	Información de algunos módulos de hardware
<code>/proc/filesystems</code>	Sistemas de ficheros habilitados en el <i>kernel</i>
<code>/proc/ide</code>	Directorio de información del bus IDE, características de discos
<code>/proc/interrupts</code>	Mapa de interrupciones hardware (IRQ) utilizadas
<code>/proc/ioports</code>	Puertos de E/S utilizados
<code>/proc/meminfo</code>	Datos del uso de la memoria
<code>/proc/modules</code>	Módulos del <i>kernel</i>
<code>/proc/net</code>	Directorio con toda la información de red
<code>/proc/pci</code>	Dispositivos pci del sistema
<code>/proc/scsi</code>	Directorio de dispositivos SCSI, o IDE emulados por SCSI
<code>/proc/version</code>	Versión y fecha del <i>kernel</i>

3.4.3 Procesos

Los procesos que se encuentren en ejecución en un determinado momento serán, en general, de diferente naturaleza. Podemos encontrar:

- Procesos de sistema, o bien procesos asociados al funcionamiento local de la máquina y del *kernel*, o bien procesos (denominados *daemons*) asociados al control de diferentes servicios, ya sean locales, o de red, porque estamos ofreciendo el servicio (actuamos de servidor) o estamos recibiendo el servicio (actuamos de clientes). La mayoría de estos procesos aparecerán asociados al usuario *root*, puede haber algunos servicios asociados a otros usuarios de sistema (*lp*, *bin*, *www*, *mail*, etc.), estos son usuarios "virtuales" que utiliza el sistema para ejecutar ciertos procesos.
- Procesos del usuario administrador: en caso de actuar como *root*, nuestros procesos interactivos o aplicaciones lanzadas también aparecerán como procesos asociados al usuario *root*.
- Procesos de usuarios del sistema: asociados a la ejecución de sus aplicaciones, ya sea tareas interactivas en modo texto o en modo gráfico.

Como comandos rápidos y más útiles podemos utilizar:

- *ps*: el comando estándar, lista los procesos con sus datos de usuario, tiempo, identificador de proceso y línea de comandos usada.
- *top*: una versión que nos da una lista actualizada a intervalos.
- *kill*: nos permite eliminar procesos del sistema mediante el envío de señales al proceso.

3.4.4 Trabajos no interactivos

En las tareas de administración, suele ser necesaria la ejecución a intervalos de ciertas tareas, ya sea por programar las tareas para realizarlas en horarios menos usados, o bien por la propia naturaleza periódica de las tareas.

Para realizar este tipo de trabajos "fuera de horas", como servicios periódicos o programados, hay diversos sistemas que nos permiten construir un tipo de agenda de tareas:

- *nohup* es quizás el caso más simple utilizado por los usuarios, les permite la ejecución de una tarea no interactiva una vez hayan salido de su cuenta. Normalmente, al salir de la cuenta, el usuario pierde sus procesos; *nohup* permite dejarlos corriendo, a pesar de que el usuario se desconecte.
- *at* nos permite lanzar una acción para más tarde, programando un determinado instante en el que va a iniciarse, especificándose la hora (hh:mm) y fecha, o bien si se hará hoy (today) o mañana (tomorrow).
- *cron* permite establecer una lista de trabajos por realizar con su programación; esta configuración se guarda en */etc/crontab*; concretamente, en cada entrada de este fichero tenemos: minutos y hora en que se va a efectuar la tarea, qué día del mes, qué mes, qué día de la semana, junto con qué (ya sea una tarea, o bien un directorio donde estarán las tareas a ejecutar).

3.4.5 Logs del sistema

Tanto el *kernel* como muchos de los *daemons* de servicios, así como diferentes aplicaciones o subsistemas de GNU/Linux, pueden generar mensajes que vayan a parar a ficheros *log*, ya sea para tener una traza de su funcionamiento, o bien para detectar errores o advertencias de malfuncionamiento o situaciones críticas. Este tipo de *logs* son imprescindibles en muchos casos para las tareas de administración y se suele emplear bastante tiempo de administración en el procesamiento y análisis de sus contenidos.

Un *daemon* particular del sistema (importante) es el *daemon* Syslogd. Este *daemon* se encarga de recibir los mensajes que se envían por parte del *kernel* y otros *daemons* de servicios y los envía a un fichero *log* que se encuentra en */var/log/messages*.

3.4.6 Memoria

Respecto a la memoria del sistema, tendremos que tener en cuenta que disponemos: a) de la memoria física de la propia máquina, b) memoria virtual, que puede ser direccionada por los procesos. Se utilizará una zona de intercambio (*swap*) sobre disco que puede implementarse como un fichero en el sistema de archivos, pero es más habitual encontrarla como una partición de intercambio (llamada de *swap*), creada durante la instalación del sistema.

Para examinar la información sobre memoria tenemos varios métodos y comandos útiles:

- Fichero */etc/fstab*: aparece la partición *swap* (si existiese). Con un comando *fdisk* podemos averiguar su tamaño.
- Comando *free*: informa sobre el estado global de la memoria. Da también el tamaño de la memoria virtual.
- Comando *vmstat*: informa sobre el estado de la memoria virtual, y el uso que se le da.

3.4.7 Discos y filesystems

Para crear una partición y asociarla a un determinado *filesystem* accesible, tendremos que realizar un proceso de montaje, ya sea explícitamente o bien programada en arranque. En el proceso de montaje se conecta el sistema de archivos asociado a la partición a un punto del árbol de directorios.

Para conocer los discos (o dispositivos de almacenamiento) que tenemos en el sistema, podemos basarnos en la información de arranque del sistema (*dmesg*), donde se detectan los presentes

Para examinar la estructura de un dispositivo conocido, o cambiar su estructura particionando el disco, podemos utilizar el comando *fdisk*

De los discos y particiones de que dispongamos, algunos de ellos se encontrarán montados en nuestro sistema de ficheros, o estarán preparados para montarse bajo demanda, o bien montarse en el momento en que dispongan de medio (en el caso de dispositivos extraíbles).

Esta información la podemos obtener de diferentes maneras.

Fichero */etc/fstab*. Indica dispositivos que están preparados para montarse en el arranque o los extraíbles que podrán ser montados. No tienen por qué estar todos los del sistema, sino sólo aquellos que queramos tener en arranque. Los demás podemos montarlos bajo demanda con el comando *mount*, o desmontarlos con *umount*.

3.4.8 Estructura del sistema de archivos de Linux

El sistema de archivo de **Linux** sigue todas las convenciones de **Unix**, lo cual significa que tiene una estructura determinada, compatible y homogénea con el resto de los sistemas **Unix**. Al contrario que en **Windows** el sistema de archivos en cualquier sistema **Unix** no está ligado de una forma directa con la estructura del hardware, esto es, no depende de si un determinado ordenador tiene 1, 2 o 7 discos duros para crear las unidades **c:**, **d:** o **m:**.

Todos los sistemas de archivos de **Unix** tienen un origen único la raíz o root representada por **/**. Bajo este directorio se encuentran todos los ficheros a los que puede acceder el sistema operativo. Estos ficheros se organizan en distintos directorios cuya misión y nombre son estándar para todos los sistemas **Unix**.

/ Raíz del sistema de archivos.

/dev Contiene ficheros del sistema representando los dispositivos que estén físicamente instalados en el ordenador.

CONTENIDO

/etc Este directorio está reservado para los ficheros de configuración del sistema. En este directorio no debe aparecer ningún fichero binario (programas). Bajo este deben aparecer otros dos subdirectorios:

/etc/X11 Ficheros de configuración de X **Windows**

/etc/skel Ficheros de configuración básica que son copiados al directorio del usuario cuando se crea uno nuevo.

/lib Contiene las librerías necesarias para que se ejecuten los programas que residen en **/bin** (no las librerías de los programas de los usuarios).

/proc Contiene ficheros especiales que o bien reciben o envían información al **kernel** del sistema (Se recomienda no modificar el contenido de este directorio y sus ficheros).

/sbin Contiene programas que son únicamente accesibles al superusuario o **root**.

/usr Este es uno de los directorios más importantes del sistema puesto que contiene los programas de uso común para todos los usuarios. Su estructura suele ser similar a la siguiente:

/usr/X11R6 Contiene los programas para ejecutar X **Window**.

/usr/bin Programas de uso general, lo que incluye el compilador de C/C++.

/usr/doc Documentación general del sistema.

/usr/etc Ficheros de configuración generales.

/usr/include Ficheros de cabecera de C/C++ (.h).

/usr/info Ficheros de información de GNU.

/usr/lib Librerías generales de los programas.

/usr/man Manuales accesibles con el comando **man** (ver más adelante).

/usr/sbin Programas de administración del sistema.

/usr/src Código fuente de programas.

Existen además de los anteriores otros directorios que se suelen localizar en el directorio

/usr, como por ejemplo las carpetas de los programas que se instalen en el sistema.

/var Este directorio contiene información temporal de los programas (lo cual no implica que se pueda borrar su contenido, de hecho,

3.4.8.1 Permisos

Otro tema que habrá que controlar en el caso de los ficheros y directorios es el de los permisos que queremos establecer en cada uno de ellos, recordando que cada fichero puede disponer de la serie de permisos: *rw-rw-rw-* que luego detallaremos más profundamente.

3.4.9 Servidores de impresión

El sistema de impresión de GNU/Linux [está heredado de la variante BSD de UNIX; este sistema se denominaba LPD (*Line Printer Daemon*). Éste un sistema de impresión muy potente, ya que integra capacidades para gestionar tanto impresoras locales como de red. Y ofrece dentro del mismo, tanto el cliente como el servidor de impresión.

- BSD LPD: es uno de los estándares de UNIX, y algunas aplicaciones asumen que tendrán los comandos y el sistema de impresión disponibles. El sistema LPD es utilizable, pero no muy configurable, sobre todo en el control de acceso, por eso las distribuciones se han movido a los otros dos sistemas más modernos.
- LPRng: se diseñó básicamente para ser un reemplazo del BSD, por lo tanto, la mayor parte de la configuración es parecida y únicamente difiere en algunos ficheros de configuración.
- CUPS: se trata de una desviación mayor del BSD original, y la configuración es propia. Se proporciona información a las aplicaciones sobre las impresoras disponibles (también en LPRng). En CUPS tanto el cliente como el servidor tienen que disponer de software CUPS.

3.5 Administración de seguridad

3.5.1 Usuarios y grupos

Los usuarios de un sistema GNU/Linux disponen normalmente de una cuenta asociada con sus datos, junto con el espacio en disco para que puedan desarrollar sus archivos y directorios. Este espacio está asignado al usuario, y sólo puede ser usado por éste (a no ser que los permisos especifiquen cosas diferentes).

Dentro de las cuentas asociadas a usuarios podemos encontrar diferentes tipos:

- La del administrador, con identificador *root*, que sólo es (o debería ser) utilizada para las operaciones de administración. El usuario *root* es el que dispone de más permisos y acceso completo a la máquina y a los archivos de configuración. Por lo tanto, también es el que más daño puede causar por errores u omisiones. Es mejor evitar usar la cuenta de *root* como si fuese un usuario más, por lo que se recomienda dejarla sólo para operaciones de administración.
- Cuentas de usuarios: las cuentas normales para cualquier usuario de la máquina tienen los permisos restringidos al uso de ficheros de su cuenta, y a algunas otras zonas particulares (por ejemplo, los temporales en */tmp*), así como a utilizar algunos dispositivos para los que se le haya habilitado.
- Cuentas especiales de los servicios: *lp*, *news*, *wheel*, *www-data*, ... cuentas que no son usadas por personas, sino por servicios internos del sistema, que los usa bajo estos nombres de usuario. Algunos de los servicios también son usados bajo el nombre de *root*.

Un usuario normalmente se crea mediante la especificación de un nombre (o identificador de usuario), una palabra de paso (*password*) y un directorio personal asociado (la cuenta).

La información de los usuarios del sistema está incluida en los siguientes archivos:

```
/etc/passwd
/etc/shadow
/etc/group
/etc/gshadow
```

3.5.2 IDs de presentación y contraseñas

El corazón del esquema de seguridad de UNIX-LINUX es el Id de presentación y la contraseña del usuario individual. Puesto que la contraseña se almacena de forma cifrada, ni tan siquiera el administrador es capaz de determinar cuál es.

La herramienta que permite modificar la contraseña es **passwd**.

Existen reglas que describen una contraseña aceptable como:

- La longitud de la contraseña es de al menos 8 caracteres y uno de ellos es un carácter numérico.
- Conveniencia de mezcla de caracteres en mayúsculas, minúsculas y símbolos.
- Cualquier secuencia inusual o no intuitiva de caracteres
- No son aceptables como contraseñas: Id de presentación, el nombre del usuario, etc.
- En general se está tratando de crear cultura de utilización de passphrases en vez de password, es decir la utilización de frases completas con o sin espacios que aumenten considerablemente el número de caracteres utilizados en las contraseñas. (visitar web howsecureismypassword.com)

3.5.3 El superusuario (root)

Cada usuario normal está restringido a sus propios ficheros y datos, y a los de su grupo. Sin embargo, *root* tiene acceso completo de lectura, escritura y ejecución a todos los ficheros y directorios del sistema. Este usuario es conocido como superusuario.

El usuario root tiene la capacidad de producir un gran daño al software y a los ficheros del sistema. Este Id debe utilizarse con sumo cuidado y exclusivamente para tareas de administración, debiendo utilizarse otro Id para las tareas rutinarias con la máquina.

El shell proporciona un inductor especial para recordar que se está trabajando como superusuario: **#**. El resto del entorno del sistema también es diferente en comparación con el de un usuario normal e incluso ciertas órdenes actúan de forma diferente cuando son emitidas por el superusuario, ya que proporcionan capacidades ampliadas.

Aparte del superusuario, también existen agentes para la administración del sistema, y se encargan de las tareas rutinarias: operaciones, administración y mantenimiento. El root puede delegar parte de sus privilegios a estos agentes cuando los crea.

3.5.4 Fichero de contraseñas

La información crítica que controla las identificaciones de los usuarios está mantenida en el fichero de base de datos */etc/passwd*, legible por todos los usuarios pero no es modificable ya que en caso contrario la seguridad se quebraría fácilmente.

Cada usuario tiene una línea en el fichero de contraseñas, y también son necesarios varios ID de presentación estándar globales del sistema para el correcto funcionamiento del mismo.

CONTENIDO

Este fichero contiene el Id del usuario, el código numérico del Id del usuario, el código numérico del Id del grupo, un comentario que contiene la dirección del usuario, el directorio propio del usuario y el nombre del camino completo de presentación del usuario.

Existe también un segundo fichero en el sistema para contener la contraseña cifrada y algunos otros datos. Este fichero es el */etc/shadow* que es legible para root.. Este fichero contiene los Ids del usuario, sus contraseñas cifradas, un código numérico que describe cuando fue modificada por última vez la contraseña y el número mínimo y máximo de días requerido entre días entre cambios de contraseña. Este fichero tampoco es modificable

Respecto a la administración de usuarios y grupos, lo que hemos comentado aquí hace referencia a la administración local de una sola máquina. En sistemas con múltiples máquinas que comparten los usuarios suele utilizarse otro sistema de gestión de la información de los usuarios. Estos sistemas, denominados genéricamente sistemas de información de red, como NIS, NIS+ o LDAP, utilizan bases de datos para almacenar la información de los usuarios y grupos, de manera que se utilizan máquinas servidoras, donde se almacena la base de datos, y otras máquinas clientes, donde se consulta esta información. Esto permite tener una sola copia de los datos de los usuarios, y que éstos puedan entrar en cualquier máquina disponible.

Podemos comprobar si estamos en un entorno de tipo NIS si en las líneas *passwd* y *group* del archivo de configuración */etc/nsswitch.conf* aparece *compat*, si estamos trabajando con los ficheros locales, o bien *nis* o *nisplus* según el sistema con que estemos trabajando.

3.5.5 Protección de datos frente a otros usuarios

Los ficheros del sistema de archivos tienen tres niveles de permisos:

- Los del usuario individual
- Los del grupo al que el usuario pertenece
- Los de los demás usuarios de la máquina

Un fichero tiene tres grupos de permisos para cada uno de los tres niveles de seguridad: Lectura, escritura y ejecución para el propietario, para el grupo y para todos los usuarios (world). Cuando se crea un nuevo fichero, el usuario que lo crea es el propietario del fichero, y su grupo se asigna como identificador (ID) del grupo. La protección de los archivos en UNIX se maneja por medio de una cadena de nueve caracteres. Los nueve caracteres se dividen en tres grupos de tres caracteres cada uno

Propietario	Grupo	Resto (world)
rwX	rwX	rwX

r: Lectura
w: Escritura
x: Ejecución

Se puede ceder la propiedad del fichero con la orden "chown" y la del grupo con la orden "chgrp".

Sólo el superusuario o root puede cambiar los permisos de cualquier fichero del sistema

3.5.6 Cifrado de ficheros

Se puede proteger los ficheros que necesiten tratamiento especial cifrándolos. Los editores como edit, vi, emacs, proporcionan la capacidad de crear y editar ficheros cifrados. Se puede decir al editor que descifre un fichero cuando lo cargue y que lo cifre cuando lo almacene en disco.

La orden `crypt` lee la entrada estándar y escribe en la salida estándar. Si la entrada está en texto llano, la salida será cifrada. Si la entrada está cifrada la salida será descifrada.

Tiene un estándar de cifrado **DES de 56 bits**.

Otra herramienta que esta mejor reconocida es el **PGP** famosa por el correo electrónico. Esta se basa en sistemas de claves privadas y públicas. Ofrece un nivel de seguridad superior al **crypt**. Se basa en un algoritmo simétrico **IDEA** de clave secreta. Después de añadieron mejoras con el **triple DES** y **CAST5**.

La mejor opción si está buscando la **privacidad** del sistema de ficheros es que el sistema de cifrado se encuentre integrado en el sistema y por lo tanto todo el disco duro este cifrado, como lo que proporcionan sistemas como TCFS y CFS.

3.5.7 Seguridad en Linux

El salto tecnológico de los sistemas de escritorio aislados, hasta los sistemas actuales integrados en redes locales e Internet, ha traído una nueva dificultad a las tareas habituales del administrador: el control de la seguridad de los sistemas.

La seguridad es un campo complejo, en el cual se mezclan tanto técnicas de análisis, con otras de detección o de prevención de los posibles ataques, como el análisis de factores "psicológicos", como el comportamiento de los usuarios del sistema o las posibles intenciones de los atacantes.

Los ataques pueden provenir de muchas fuentes y afectar desde a una aplicación o servicio, hasta a algún usuario, o a todos, o al sistema informático entero.

La seguridad en administración puede ser entendida como el proceso que ha de permitir al administrador del sistema prevenir y detectar usos no autorizados de éste. Las medidas de prevención ayudan a parar los intentos de usuarios no autorizados (los conocidos como *intrusos*), para acceder a cualquier parte del sistema. La detección ayuda a descubrir cuándo se produjeron estos intentos o, en el caso de llevarse a cabo, establecer las barreras para que no se repitan y poder recuperar el sistema si éste ha sido quebrantado.

Veamos algunos tipos de medidas que podríamos tomar en los campos de prevención y detección de intrusos.

3.5.7.1 Seguridad local en Linux

- Uno de los problemas ya está en el arranque del sistema. Si el sistema puede arrancarse de disco o CD, un atacante podría acceder a los datos de una partición Linux (o también Windows) tan sólo con montar el sistema de ficheros y podría colocarse como usuario *root* sin necesidad de conocer ninguna contraseña. En este caso, se necesita proteger el arranque del sistema desde la BIOS, por ejemplo, protegiendo el acceso por contraseña, de manera que no se permita el arranque desde CD o disquete.

El siguiente paso es proteger el *bootloader*, ya sea *lilo* o *grub*, para que el atacante no pueda modificar las opciones de arranque del *kernel* o modificar directamente el arranque (caso de *grub*). Cualquiera de los dos puede protegerse también por contraseñas.

En *grub*, el fichero `/sbin/grub-md5-crypt` pide la contraseña y genera una suma *md5* asociada. Después, el valor obtenido se introduce en

```
/boot/grub/grub.conf. Bajo la línea timeout, se introduce  
password --md5 suma-md5-calculada
```

Otro tema relacionado con el arranque es la posibilidad de que alguien que tenga acceso al teclado reinicie el sistema, debido a que si se pulsa CTRL+ALT+DEL, se provoca una operación de *shutdown* en la máquina. Este comportamiento viene definido en `/etc/inittab`, con una línea como

```
ca:12345:ctrlaltdel:/sbin/shutdown -t1 -a -r now
```

- Otras posibles actuaciones consisten en obligar a los usuarios a cambiar la contraseña con frecuencia (el comando `change` puede ser útil), imponer restricciones en el tamaño y el contenido de las contraseñas, y validarlas contra diccionarios de términos comunes.

Respecto a las herramientas, es interesante disponer de un *cracker* de contraseñas (o sea, un programa para obtener contraseñas), para comprobar la situación real de seguridad de las cuentas de nuestros usuarios, y forzar así el cambio en las que detectemos inseguras. Dos de las más utilizadas por administradores son John the Ripper, "crack" o slurpie.

- Otro problema importante son algunos permisos especiales que son utilizados sobre ficheros o *script*.

El bit *sticky* se utiliza sobre todo en directorios temporales en que queremos que en algunos grupos (a veces no relacionados), cualquier usuario pueda escribir, pero sólo pueda borrar bien el propietario del directorio, o bien el propietario del fichero que esté en el directorio. Un ejemplo clásico de este bit, es el directorio temporal `/tmp`. Hay que vigilar que no haya directorios de este tipo, ya que pueden permitir que cualquiera escriba en ellos, por lo que habrá que comprobar que no haya más que los puramente necesarios como temporales. El bit se fija con el comando (`chmod +t dir`), y puede quitarse con `-t`. En un `ls` aparecerá como un directorio con permisos `drwxrwxrwt` (fijaos en la última `t`).

El bit *setuid* permite a un usuario ejecutar (ya sea un ejecutable o un *shell script*) con los permisos de otro usuario (como por ejemplo de root). El bit se coloca mediante `chmod +s`. En el caso de visualizar con `ls`, el fichero aparecerá con `-rwsrw-rw` (fijaos en la `S`).

- Los módulos PAM son un método que permite al administrador controlar cómo se realiza el proceso de autenticación de los usuarios para determinadas aplicaciones. Los módulos PAM (las bibliotecas) suelen estar en el directorio `/lib/security`. Y la configuración de PAM está presente en el directorio `/etc/pam.d`, donde aparece un fichero de configuración de PAM por cada aplicación que está usando módulos PAM.

3.5.7.2 Seguridad en red en Linux

Como clientes de servicios, básicamente tenemos que asegurar que no pongamos en peligro a nuestros usuarios (o se pongan en peligro solos) por usar servicios inseguros. Evitar el uso de servicios que no utilicen cifrado de datos y contraseñas (ftp, telnet, correo no seguro). Utilizar en este caso técnicas de conexión cifradas, como SSH y SSL.

Otro punto importante se refiere a la posible sustitución de servidores por otros falsos, o bien a técnicas de secuestro de sesiones. En estos casos tenemos que disponer de mecanismos de autenticación potentes que nos permitan verificar la autenticidad de los servidores (por ejemplo SSH y SSL tienen algunos de estos mecanismos).

La configuración de los servicios de red se realiza desde varios lugares.

- En `/etc/inetd.conf` o el equivalente directorio `/etc/xinetd.d`: estos sistemas son una especie de "superservidores", ya que controlan varios servicios hijos y sus condiciones de arranque.
- Servidores iniciados en arranque: según el *runlevel* tendremos una serie de servicios arrancados. El arranque se originará en el directorio asociado al *runlevel*, por ejemplo, en Debian el *runlevel* por defecto es el 2, los servicios arrancados serán arrancados desde el directorio `/etc/rc2.d`, seguramente con enlaces a los *scripts* contenidos en `/etc/init.d`, en los cuales se ejecutará con el parámetro *start*, *stop*, *restart*
- Otros servicios de tipo RPC: asociados a llamadas remotas entre máquinas son, por ejemplo, utilizados en NIS y NFS. Puede examinarse cuáles hay con el comando `rpcinfo -p`.
- Otros ficheros de apoyo (con información útil) son: `/etc/services`, que está formado por una lista de servicios locales o de red conocidos, junto con el nombre del protocolo (tcp, udp u otros), que se utiliza en el servicio, y el puerto que utiliza; `/etc/protocols` es una lista de protocolos conocidos; y `/etc/rpc` es una lista de servidores RPC, junto con los puertos usados.

Con los sistemas de detección de intrusos (IDS) se quiere dar un paso más adelante. Una vez hayamos podido configurar más o menos correctamente nuestra seguridad, el paso siguiente consistirá en una detección y prevención activa de las intrusiones. Algunos ejemplos de herramientas

CONTENIDO

IDS serían: Logcheck (verificación de *logs*), TripWire (estado del sistema mediante sumas *md5* aplicadas a los archivos), AIDE (una versión libre de TripWire), Snort (IDS de verificación de estado de una red completa).

Los TCP wrappers son un software que actúa de intermediario entre las peticiones del usuario de servicio y los *daemons* de los servidores que ofrecen el servicio. Muchas de las distribuciones vienen ya con los *wrappers* activados y podemos configurar los niveles de acceso. Los *wrappers* se suelen utilizar en combinación con *inetd* o *xinetd*, de manera que protejan los servicios que ofrecen

Un *firewall* (o cortafuegos) es un sistema o grupo de sistemas que refuerza las políticas de control de acceso entre redes. El *kernel* Linux (a partir 2.4) proporciona un subsistema de filtrado denominado Netfilter que proporciona características de filtrado de paquetes y también NAT (Network Address Translation). Este sistema permite usar diferentes interfaces de filtrado, entre ellas, la más usada se denomina IPtables. El comando principal de control es *iptables*. Algunas herramientas incluidas con las distribuciones son:

- *lokkit*: en Red Hat, muy básico. También existe una versión gráfica llamada *gnome-lokkit*.
- Bastille: programa de seguridad bastante completo y didáctico
- *fwbuilder*: una herramienta que permite construir las reglas del *firewall* de forma gráfica. Se puede utilizar en varios operativos (GNU/Linux tanto Red Hat como Debian, OpenBSD, MacOS), con diferentes tipos de *firewalls* (*iptables* incluido).

3.5.7.3 Herramientas de Seguridad

A continuación, comentamos brevemente algunas herramientas, así como algunos usos que pueden utilizar:

- TripWire: esta herramienta mantiene una base de datos de sumas de comprobación de los ficheros importantes del sistema. Puede servir como sistema IDS preventivo. Nos sirve para “tomar” una foto del sistema, poder comparar después cualquier modificación introducida y comprobar que éste no haya sido corrompido por un atacante
- Nmap: es una herramienta de escaneo de puertos para redes grandes. Puede escanear desde máquinas individuales a segmentos de red.
- Ethereal: es un analizador de protocolos y captura el tráfico de la red (es un *sniffer*).
- Snort: es un sistema IDS que permite realizar análisis de tráfico en tiempo real y guardar *logs* de los mensajes. Permite realizar análisis de los protocolos, búsquedas por patrones (protocolo, origen, destino, etc.).
- Nessus: es un detector de vulnerabilidades conocidas y asesora sobre cómo mejorar la seguridad para las detectadas.

3.5.7.4 Logs

Dando una visión sobre los ficheros de *logs* podemos hacernos una idea rápida del estado global del sistema, así como de las últimas ocurrencias, y detectar accesos (o intentos) indebidos. Hay que tener cuidado, ya que los *logs*, si ha habido una intrusión real, pueden haber sido limpiados o falseados. La mayoría de archivos de *log* residen en el directorio */var/log*.

Muchos de los servicios pueden poseer *logs* propios, que normalmente se establecen en la configuración (mediante el correspondiente fichero de configuración). La mayoría suelen utilizar las facilidades de *log* incorporadas en el sistema Syslog, mediante el demonio Syslogd. Su configuración reside en */etc/syslog.conf*. Esta configuración suele establecerse por niveles de mensajes: existen diferentes tipos de mensajes según su importancia. Normalmente suelen aparecer niveles *debug*, *info*, *err*, *notice*, *warning*, *err*, *crit*, *alert*, *emerg*, en que más o menos ésta sería la ordenación de importancia de los mensajes (de menor a mayor). Normalmente, la mayoría de los mensajes se dirigen al *log* */var/log/messages*, pero puede definirse que cada tipo vaya a ficheros diferentes y también se puede identificar quién los ha originado; típicamente el *kernel*, correo, *news*, el sistema de autenticación, etc.

3.5.7.5 Otros aspectos de la seguridad de Linux

Linux sigue el mismo esquema de seguridad que Unix, pero dado su carácter "libre" se han desarrollado en este entorno productos que proporcionan capacidades adicionales y herramientas a los especialistas en seguridad que las suministradas por defecto. Entre los numerosos productos que existen hoy en día en el mercado podemos destacar:

- Kali Linux
- Back Box
- ArchAssault
- Knoppix STD

Kali Linux: es una versión personalizada de Debian con un portfolio inmenso de herramientas de penetración o análisis forense.

Knoppix STD (Security Tools Distribution) es una versión personalizada de Knoppix, una distribución de Linux pensada para ser ejecutada directamente desde el CD-ROM ("Live CD"). Cuando se arranca la máquina con Knoppix STD, no se realiza ningún tipo de modificación en la configuración del ordenador.

Knoppix STD incluye un gran número de herramientas de interés para la seguridad, todas ellas preparadas para ser ejecutadas directamente desde el CD. Las herramientas se dividen en varias categorías:

- Autenticación, identificación de contraseñas, cifrado
- Herramientas para el análisis forense, cortafuegos, honeypots, sistemas de detección de intrusiones
- Herramientas para la gestión de redes
- Un gran número de herramientas para la realización de pruebas de penetración, sniffers
- Herramientas para la realización de valoraciones de seguridad y herramientas para la realización de pruebas de redes sin fines.

LocalAreaSecurity: Esta es otra distribución "LiveCD", de pequeño tamaño (185 MB, pensada para instalarse en un CD pequeño, de la medida de una tarjeta de crédito). También está basada en Knoppix y utiliza el núcleo 2.4.20.

LocalAreaSecurity está especializada en la realización de pruebas de verificación de la seguridad y en las pruebas de penetración, incluyendo un gran número de herramientas especializadas: sniffers, cifrado, monitorización de redes, detección de información oculta, obtención de información, etc.

R.I.P. (Recovery Is Posible) Linux: Se trata de una distribución de Linux pensada por recuperar datos de sistemas de ficheros defectuosos. Merced a esta distribución, se puede recuperar los datos de una máquina con un disco duro defectuoso que Windows se negaba a reconocer ni tampoco sabía como reparar. Con R.I.P. Linux fue posible no tan solo montar y acceder a la información, sino transferirla por la red a otro sistema.

Es un sistema para arranque desde CD o unidad de disco, para rescate, respaldo o mantenimiento basado en Slackware. Tiene soporte para muchos tipos de sistemas de archivos (Reiserfs, Reiser4, ext2/3, iso9660, UDF, XFS, JFS, UFS, HPFS, HFS, MINIX, MS DOS, NTFS, y VFAT) y contiene un montón de herramientas para recuperación del sistema. También da soporte a IDE/SCSI/SATA, PCMCIA, RAID, LVM2, así como para redes Ethernet, DSL, de cable, PPP y PPPOE.

WARLINUX: Esta distribución de Linux, en modo texto, está especialmente pensada para la verificación de la seguridad de las redes inalámbricas. Funciona directamente desde el CD y permite identificar las redes inalámbricas que están al alcance del ordenador y la realización de auditorios de seguridad y valoración de su nivel de seguridad.

FIRE: Esta versión de Linux incluye las herramientas necesarias para la realización de valoraciones de seguridad, respuesta a incidentes de seguridad, pruebas de penetración y análisis forense de sistemas y recuperación de datos en sistemas Windows, Solaris (SPARC) y Linux (x86). Adicionalmente, FERIO incluye un programa para la detección de virus (F-Prot).

3.6 Administración de red

El sistema operativo UNIX (GNU/Linux) es tomado como ejemplo de una arquitectura de comunicaciones estándar. Desde el mítico UUCP (servicio de copia entre sistemas operativos UNIX) hasta las redes actuales, UNIX siempre ha mostrado su versatilidad en aspectos relacionados a la comunicación y el intercambio de información. Con la introducción de redes de ordenadores (área local LAN, área amplia WAN o las más actuales área metropolitana MAN) ofreciendo enlaces multipunto a diferentes velocidades (56kbts/seg hasta 10Gbit/seg), han ido surgiendo nuevos servicios basados en protocolos más rápidos, portables entre diferentes ordenadores y mejor adaptados, como el TCP/IP (Transport Control Protocol)

Tres clases generales de software los diseños de LAN. El sistema de ficheros por red (**Network File System NFS**) está orientado principalmente hacia el entorno Ethernet, y el sistema compartición ficheros remotos (**Remote File Sharing – RFS**) está orientado hacia StarLan. Sin embargo, actualmente es posible cualquier arquitectura LAN con cualquier sistema de software, aunque esto suele hacerse raramente. Una tercera clase, las órdenes de acceso remoto, es transparente al tipo de red y no hacen distinción entre ellas.

Muchos de los servicios de red requieren que haya ciertos procesos demonios ejecutándose en la máquina servidora. Estos demonios son típicamente inicializados cuando la máquina se arranca con init 3. Además de estos demonios normales, algunas de las órdenes de acceso remoto requieren demonios adicionales.

3.6.1 NFS (Network File System)

NFS se basa en la idea de permitir que una colección arbitraria de clientes y servidores, compartan un mismo sistema de archivos. NFS puede ejecutarse entre ordenadores situados en redes distintas e incluso en un solo ordenador, que ejecuta las funciones de servidor y cliente al mismo tiempo. NFS implementa sus propios protocolos de comunicación entre los clientes y los servidores. En resumen, los equipos servidores exportan una lista de subdirectorios a compartir, lo que incluye toda la jerarquía que tienen por debajo y los clientes conectan esos recursos a sus propias jerarquías haciendo el acceso completamente homogéneo y transparente al usuario.

Una vez conectado el sistema físicamente a la red, hay que configurar el sistema NFS. Para ello se necesitan tres datos fundamentales:

- Dirección Internet de la máquina
- Dirección física
- Nombre del host.

La dirección Internet es un número de cuatro partes que define la máquina con una dirección lógica que debería de ser única en todo el mundo. Esta dirección es asignada por el administrador de acuerdo aun esquema jerárquico de numeración mundial.

La dirección física hardware viene determinada por el hardware del controlador Ethernet, y también es única en el mundo. Ésta es visualizada cuando se arranca la máquina

Todas las órdenes de comunicación por red utilizan nombres de máquinas lógicos que son lo nombres hosts, aunque internamente la máquina utilice un esquema de direccionamiento numérico cuando se comunica con otras. Esta dirección numérica es la dirección Internet. El fichero `/etc/inet/hosts` contiene la correspondencia entre las dos formas.

3.6.2 RFS (Remote File Sharing)

Para utilizar RFS hay que configurar un dominio y la aparición de la máquina en el dominio. Un dominio es una partición separada de una red mayor, y está controlado por demonios que corren en un servidor de nombres para ese dominio. Si se está comenzando un nuevo dominio, la máquina puede ser un servidor de nombres primario o secundario, o una máquina normal que puede

compartir recursos y montarlos. Se puede limitar la máquina a un dominio específico de la red, o proteger el acceso a los recursos con contraseñas.

Un dominio es un grupo de máquinas que permiten comunicación entre ellas mismas, pero también pueden excluir a máquinas que no son miembros del dominio. Las redes NFS también pueden tener dominios que están controlados por el administrador de red

Cuando se establece un nuevo dominio, debe prepararse en primer lugar el servidor de nombres primario. Un dominio sólo debe tener un servidor de nombres primario, aunque puede tener varios servidores de nombres secundarios. El servidor de nombres primario ejecuta demonios administrativos que gestionan el dominio y efectúan chequeos de seguridad. Los servidores de nombre secundarios se encargan de estas tareas cuando el servidor de nombres primario está inactivo.

Estas facilidades de comunicación por red de bajo nivel de UNIX / LINUX permiten el desarrollo de pilas de protocolos, que implementan la comunicación efectiva a través de una red. El más popular es TCP/IP (Transmisión Control Protocol / Internet Protocol) que es utilizado con el sistema NFS

3.7 Herramientas Integradas de Administración

Se pueden manejar la mayoría de las tareas rutinarias de administración del sistema a través del agente de usuario proporcionado por los vendedores. Las versiones de UNIX - Linux proporcionan el sistema OA&M: Operación, administración y mantenimiento.

En la mayoría de las versiones, esta herramienta es el sysadm (adm), debe ser ejecutada desde el directorio raíz y hay que poseer privilegios de root.

Este agente de usuario estándar: el **sistema OA&M** está basado en las herramientas **FMLI**: Intérprete de Lenguajes de formularios y Menús (Form and Menu Language Interpreter).

Esta herramienta permite gestionar usuarios, grupos y permisos; copias de seguridad; configuración de impresoras, etc. Además, los propios fabricantes proporcionan las suyas que incluso permiten modificar particiones de disco o añadir mas espacio a una partición de un disco.

4. Interfaz usuario

La interfaz de usuario sistema – operativo (usuario – máquina) es la forma y utilidades que facilitan al usuario su interacción con el sistema operativo. Existen dos maneras de interactuar con el sistema operativo: En modo comando a través de shell, o en modo gráfico a través de gestores de ventanas y escritorio.

Tradicionalmente Unix ha proporcionado el shell como interfaz por defecto. Posteriormente adoptó en sistema de ventanas X proporcionando un entorno gráfico WIMP (Windows, icons, mouse, pointer). Con la llegada de Linux y el desarrollo del software libre, se fueron implementando en las distintas distribuciones gestores de ventanas más avanzados, librerías de desarrollo y entornos de escritorio, que nada tiene que envidiar a Microsoft Windows.

En lo sucesivo vamos a ver con un poco más de detalle cada uno de estos entornos.

4.1 Interfaz modo comando: Shell

El shell es un proceso a nivel de usuario como cualquier otra orden. No tiene relación especial con el núcleo, ni privilegios especiales del que no gocen otras órdenes. El shell puede ser modificado a discreción del usuario y hay varios alternativos disponibles con aspectos diferentes o que están organizados para diferentes funciones.

La shell muestra la interfase entre UNIX y el usuario, interpretando instrucciones entre el usuario o su aplicación y el Kernel y los programas de utilidades y transformando peticiones en acciones. Por ser un programa invocado para interpretar órdenes y no una parte integral del núcleo es fácil de disponer de diferentes versiones:

El shell estándar es el conocido como el *shell de Bourne* (laboratorios Bell de ATT). Es un shell pequeño relativamente eficiente, diseñado para uso de propósito general. Inicialmente no disponía de *historia de órdenes*, es decir, no se podía repetir una línea de orden sin volverla a escribir completamente; no permitía *alias*: capacidad de los usuarios de personalizar los nombres de las órdenes que utilizan más frecuentemente. Estos dos problemas se solucionaron creando nuevas órdenes como programas shell, salvándolos en un fichero y dándole al fichero el nombre que se desee.

Posteriormente surgieron otros dos shell: *el shell C* y *el shell de Korn*, más potentes que el anterior. Estos dos proporcionan varias mejoras respecto al anterior incluyendo más operadores de programación shell, operadores aritméticos y mejores características de uso de cadenas. Además permiten:

- Edición de órdenes
- Historia de órdenes
- Alias

La *historia de órdenes* y la *edición* funcionan juntas de modo que se puede recordar una orden previamente ejecutada con la característica de historia modificándola ligeramente para una nueva invocación y luego ejecutar la orden editada como si hubiera sido vuelta a escribir. Los alias proporcionan un modo eficiente de personalizar las líneas de órdenes que se utilicen más frecuentemente.

Una vez invocada la shell correspondiente, se pueden usar características que la hacen parecida a un lenguaje de programación:

- Variables
- Estructuras de control: if, while, etc.
- Subprogramas
- Paso de parámetros
- Manejo de interrupciones

Estas características le proporcionan capacidad para diseñar herramientas propias. Los archivos de órdenes se denominan procedimientos shell, o archivos shell o guiones shell (scripts en inglés). Estos archivos generalmente tienen la extensión .sh, y empiezan con el llamado shebang que básicamente indica el shell que tendrá que interpretar ese script.

```
#!/bin/ksh
```

Todo comando o proceso lleva asociado 3 ficheros estándar: **stdin**, **stdout**, **stderr**. Es posible redireccionar el origen o el destino de estos ficheros, secuenciar tareas (;), encolar tareas de modo que la salida de una sea la entrada de la siguiente (|), ejecutar multiproceso (&).

4.1.1 El Shell Korn

El shell de Korn ofrece un conjunto mayor de características mejoradas que el shell C y la edición de órdenes y las características de historia son más fáciles de aprender y de usar. Es exactamente igual al shell de Bourne en su característica básica pero mucho más rápida y eficiente. Es un superconjunto del shell de Bourne, cualquier guión o línea del shell que sea aceptable al shell de Bourne también debería de funcionar con el de Korn.

Puede iniciarse el shell de Korn como cualquier otra orden, es decir:

```
$ ksh
```

El **ksh** almacena cada orden en una *lista de historia*. Las órdenes de la historia están numeradas secuencialmente y con un número de secuencia de cada orden visualizado por el operador "!". Por defecto en la lista de la historia se guardan las 128 órdenes más recientes.

Otra característica importante del ksh es el uso de *alias*. Los alias permiten cambiar el entorno de ksh de modo que los nombres de las órdenes que se introduzcan en la línea de orden sean modificados por ksh a otras órdenes antes de que sean ejecutadas. Se puede modificar permanentemente el repertorio de órdenes para utilizar órdenes personalizadas, pero seguir usando los nombres de órdenes normales. Por ejemplo, para la orden "**rm -i**" borrar un fichero solicitando confirmación, se puede crear el alias "**rm**" (utilizado para borrar sin confirmación) de tal forma que cuando se ejecute éste último, el ksh ejecutará "**rm -i**". Esta sustitución de alias por los nombres de orden se llama *expansión de alias*.

El ksh introduce mejoras para potenciar la programación shell. Sin embargo, éstas no son compatibles con el shell de Bourne. Las mejoras introducidas son:

- Operadores aritméticos
- Arrays
- Mejoras de solicitud de entrada y salida
- Procesamiento de variables del entorno

4.1.2 El Shell CSH

La shell-C difiere significativamente del de Bourne pero está más ampliamente disponible en diferentes versiones UNIX. Su sintaxis de programación es supuestamente parecida a la del lenguaje C. El shell C no soporta funciones shell.

El shell C se puede iniciar desde la línea de comando con la orden:

```
$ csh
```

El csh incluye soporte para la historia de órdenes y para la edición de órdenes desde la lista de historia. Sin embargo, es difícil editar un orden que esté siendo actualmente introducida, una que no se encuentra en la lista de historia. La variable “**history**” define el tamaño de la lista de historia.

El csh mantiene la lista de historia internamente, no existe un fichero de historia como en el ksh.

El csh dispone de una facilidad alias que permite que las órdenes se modificadas antes de ser ejecutadas. La orden “alias” sirve para crear un alias.

4.1.3 El Shell Bash

BASH, Bourne Again Shell, una versión mejorada del shell creado en la universidad de Bourne y que se considera un estándar dentro de Unix.

Si en este shell escribimos una palabra, el shell comprobaba primero si es un comando interno, y lo ejecutara, o después buscara, a través de una serie de directorios predefinidos un archivo con ese nombre, y si es ejecutable lo ejecutara, pasándole el control de la sesión hasta que el programa acabe y volvamos al shell en espera de otra orden.

Hay que destacar que no sirve que un programa esté en el directorio en el que estamos, ya que por seguridad, el shell no busca el programa en el directorio actual, solo en una lista de sitios (directorios) donde sabe que hay programas seguros, evitando así una de las formas de entrada de virus.

Para saber por qué sitios busca podemos escribir `set | grep PATH` que nos mostrará la lista separada por “:”.

4.2 Interfaces gráficos

La característica más importante del entorno gráfico es su completa independencia del núcleo del sistema operativo, lo que permite seleccionar diferentes interfaces para trabajar. Por otro lado, cualquier problema derivado de la interfaz no afecta a la estabilidad del sistema, siendo suficiente con terminar el programa y relanzar nuevamente la interfaz para recuperar la funcionalidad.

Inicialmente Unix solo era utilizado con terminales texto hasta que en 1984, el MIT dentro del proyecto ATHENA, crea un entorno gráfico distribuido: X Windows System independiente del sistema operativo que se llamó X-Windows. Este entorno fue mantenido por el X Consortium hasta 1996. Desde entonces se encarga The Open Group (www.X.org). La versión actual del estándar es X11, es la X11R7.7 (2012) .

Las especificaciones X-Windows definen el método por el cual se pueden comunicar las aplicaciones con el hardware gráfico. También establece un conjunto de funciones de programación bien definidas que podrán ser llamadas para realizar la manipulación básica de las ventanas.

La definición básica de cómo se dibuja una ventana y de cómo se manejan los clics del ratón no incluye la definición de cómo se deberían ver las ventanas. En realidad X-Windows en su estado natural no tiene apariencia real. El control de la apariencia se pasó a un programa externo denominado **gestor de ventanas**. El gestor de ventana se preocupa de dibujar los bordes, usar el color y hacer que el entorno sea agradable a la vista. El gestor de ventana solo se requiere para usar las llamadas estándares al subsistema X-Windows para dibujar sobre la pantalla.

Sin embargo la dificultad de programar en este entorno propició la aparición, a finales de los 90, de dos grupos independientes que aportaron soluciones a los problemas de X-Windows: **GNOME** y **KDE**. KDE ofrece un gestor de ventanas nuevo y las bibliotecas necesarias para escribir aplicaciones de un modo mucho más fácil. GNOME ofrece un marco general para el resto de gestores de ventana

y para las aplicaciones que trabajan con ellos. Cada uno tiene una idea diferente de cómo deben funcionar las cosas, pero debido a que trabajan por encima de X-Windows, no son enteramente incompatibles.

4.2.1 Arquitectura X - Windows

Es un sistema de gráficos distribuidos basado en el modelo cliente / servidor: Xserver, Xclient. Las aplicaciones gráficas se ejecutan en el servidor y los resultados se visualizan en el cliente. Por encima del servidor X está el gestor de ventanas que define la apariencia final y recoge y procesa las entradas del usuario a través del ratón, teclado, etc.

Incorporan un toolkit: Objetos gráficos (widgets) y APIs. La API incorporada está definida para los programadores, pero es difícil de usar (Xlib)

Linux utiliza una implementación libre: Xfree86 que es bastante completa y está optimizada. Aprovecha el hardware 3D mediante DRI (Direct Rendering Infrastructure).

Por encima del gestor de ventanas se situaría el gestor de escritorio, y por encima de éste los temas.

En resumen, se puede decir que tenemos un modelo de capas, donde las funciones de cada capa pueden llegar a solaparse:

Servidor X – Gestor de ventanas – Gestor de escritorio – Temas.

A la hora de gestionar un X Windows System, existe una fuerte dependencia del gestor de ventanas utilizado y del sistema de escritorio. Los tres paquetes más importantes de Linux en este campo son: fvwm, KDE y GNOME.

- fvwm es un gestor de ventanas
- KDE es un paquete de programas X que parte del gestor de escritorio incorpora un gestor de ventanas
- GNOME es otro paquete de programas X pero que no incluye gestor de ventanas

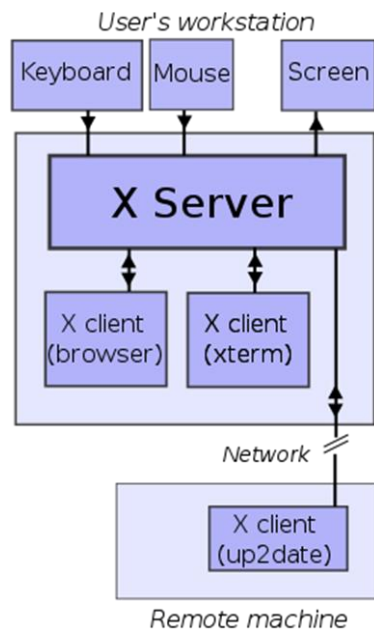


Figura: Esquema general de los componentes principales del sistema X-Windows

4.2.2 Gestor de ventanas (X Windows Manager)

Un gestor de ventanas se encarga de cómo los programas windows son visualizados en la pantalla. También ofrecen la opción de ejecutar programas a través de menús. La influencia del gestor de ventanas puede ser vista en el diseño de las mismas

Las modernas librerías para programación X soportan los temas (themes). Ésta es una opción para variar la presentación de los elementos estándar. Los themes son soportados por arios gestores de ventanas y su popularidad se debe principalmente a KDE y GNOME.

KDE utiliza como gestor de ventanas uno propio el KDE Windows Manager. GNOME no posee gestor de ventanas propio, pero puede ser usado con otros, siempre y cuando éstos se ajusten a la norma GNOME. El más popular gestor de ventanas que trabaja con GNOME es Enlightenment, que es que por defecto RedHat distribuye.

4.2.3 Entorno de escritorio

Los rasgos que caracterizan a entorno de escritorio son:

- Ofrece una interfase de usuario para opciones elementales
- Un manejo tan intuitivo como sea posible
- Incorporan paquetes de utilidades para las operaciones más frecuentemente utilizadas: editor de texto, calculadora, etc.
- Todos los componentes del escritorio pueden ser fácilmente configurados.
- Para todos los componentes o programas existe extensiva documentación ON-LINE en formato HTML.
- La transferencia y utilización de datos de diferentes programas de escritorio no es compleja
- Las funciones de Internet son transparentemente soportadas por todos los componentes. Los datos locales y externos pueden ser procesados de la misma forma.

Básicamente los usuarios de Linux estaban de acuerdo en la necesidad de un nuevo entorno de escritorio, moderno y fácil de usar. Tanto GNOME como KDE reúnen estas características. KDE (K Desktop Environment) y GNOME (GNU Network Object Model Environment) crean un escritorio moderno y fácil de usar, y lo que es más importante, son entornos libres para Linux

La discusión entre ambos paquetes o está en cual de los dos es más bonito o más funcional. De hecho, el núcleo del debate está en las librerías en las que los dos entornos están contruidos. KDE y GNOME difieren en dos puntos: En el número de funciones y en la presentación.

En el número de funciones KDE está muy por delante de GNOME. Hay más programadores de KDE que de GNOME.

La presentación constituye el principal atractivo de ambos debido a sus modernas imágenes que incluso Microsoft Windows y MacOSX admiran.



Figura: Interfaz gráfica de un entorno X-Window actual

En cuanto a las diferencias técnicas entre ambos se caracterizan por:

- Están basados en diferentes librerías
- KDE incorpora un gestor de ventanas mientras que GNOME permite que cada uno utilice el suyo, proporciona más flexibilidad en detrimento de una mayor uniformidad

KDE y GNOME son programas de aplicación sobre Linux. Ni el gestor de ventanas ni el escritorio tienen influencia sobre los programas que pueden ejecutarse en un entorno X. El único prerequisite para la ejecución de esos programas está en el conjunto de librerías necesarias que deben ser instaladas.

Tanto KDE como GNOME han inspirado el desarrollo de nuevos programas de aplicación. Los problemas surgen cuando se intenta instalar programas que no han sido suministrados en la correspondiente distribución.

Otro de los problemas es la falta de estabilidad. KDE y GNOME fueron desarrollados muy deprisa los que origina que muchos programas no tengan la misma estabilidad que sus predecesores. A pesar de esto, las distribuciones de KDE y/o GNOME han ofrecido mayor facilidad de usos que incluso Microsoft Windows

4.3 KDE (K Desktop Environment) - PLASMA

La primera versión Kde 1 es del 1988. La version actual Kde **5** es de Julio de 2014.

A los diferentes entornos de trabajo a nivel grafico o workspaces se les denomina PLASMA, como por ejemplo el Plasma Desktop o Plasma NetBook. Eso ha provocado que se popularice el nombre de PLASMA al hablar de KDE. De hecho a los pequeños widgets que es posible programar en KDE se los denomina PLASMAOIDES.

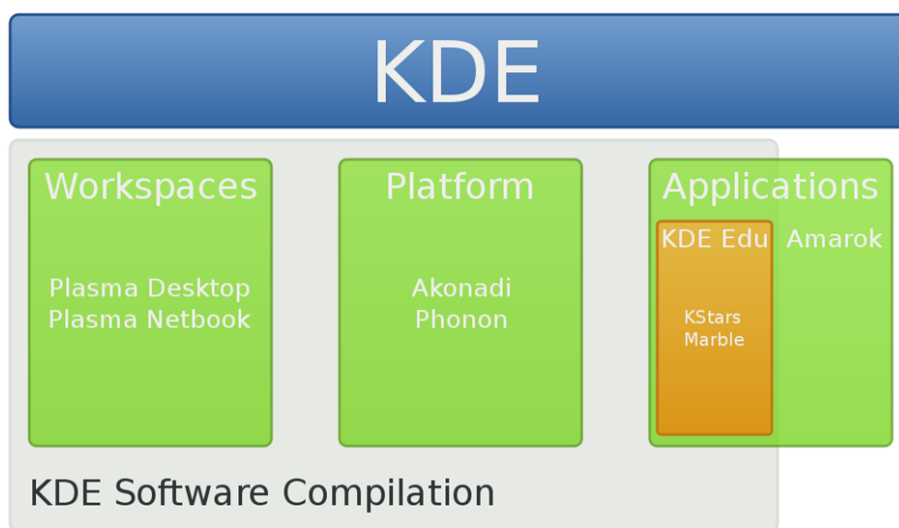


Figura: Esquema de KDE

KDE está basado en Qt, una librería comercial de la Trolltech Company. Esta librería puede ser usada libremente en proyectos de software libre; sin embargo, en comparación con licencias GPL/LGPL (GNOME está sujeto a esta licencia) tenía demasiadas restricciones. Los desarrolladores de KDE argumentaron que la distribución de Qt era insuficiente. RedHat rehusó distribuir KDE debido a los problemas con el copyright.

Esta situación supuso un peligro para KDE con lo que Trolltech anunció que Qt obtendría una licencia QPL (Q Public License). Aunque esta licencia es más restrictiva que LGPL, al menos asegura que KDE permanecerá libre en el futuro y podrá ser modificado sin permiso de Trolltech. En la actualidad KDE puede ser ejecutado tanto en Linux como en UNIX.

KDE o entorno de escritorio K, es un entorno de escritorio ligeramente distinto de los gestores típicos de ventanas. En lugar de describir cómo se ve la interfaz, KDE proporciona un conjunto de bibliotecas que, si se usan, permiten a una aplicación mejorar algunas características especiales que ofrece el gestor de ventanas (soporte de pinchar y arrastra, soporte de impresión estandarizado, etc.). KDE ofrece al programador una biblioteca mucho más sencilla de usar que el trabajo directo con la interfaz X-Windows. Una de las herramientas más potentes es el escritorio virtual, lo que permite tener varias pantallas efectivas al mismo tiempo.

Los diferentes subsistemas que componen KDE son:

- **DCOP** *Desktop Communication Protocol*: Se encarga de la gestión del protocolo de comunicaciones del escritorio.
- **KIO** *Network Transparent I/O*: Gestiona las comunicaciones dentro de un entorno de red de forma transparente para el usuario
- **SYCOCA** *SYstem COnfiguration Cache* : Se encarga de la configuración de la memoria caché
- **KParts** : Componentes integrados
- **KHTML**: librería HTML 4.0
- **XMLGU**: Arquitectura dinámica GUI basada en XML .
- **Art.**: Sistema Multimedia

Esas partes son solo un resumen somero. En la página de la wikipedia de KDE, <http://en.wikipedia.org/wiki/KDE> al final del texto aparece una desarrollada tabla con todas las tecnologías implicadas, que son muchas.

4.3.1 Elementos del KDE

Se puede considerar que los principales elementos del KDE son:

- Panel
- Centro de control
- Panel de configuración
- Ejecución de programas en un icono
- Herramientas del panel

4.3.2 Panel

El Panel (KDE panel for short o también llamado kicker) es uno de los elementos de control central de KDE. Asiste en la ejecución de programas, permite el cambio entre programas que se están ejecutando, y el cambio entre diferentes escritorios.

Centro de control KDE (KDE control Center)

Este elemento, también llamado Kcontrol, permite la configuración de componentes importantes del KDE como por ejemplo:

- Lenguaje: Idioma en el que se visualizarán todos los menús y textos de diálogos
- Tamaño de las fuentes
- Posición de las ventanas
- Salvapantallas
- Animación de los títulos de la ventanas

4.3.3 Panel de configuración

Se encarga de ajustar la apariencia del KDE panel. Estos ajustes se pueden hacer vía menús de contexto o a través del centro de control KDE. Se puede modificar desde la posición del panel KDE y la barra de herramientas hasta la forma de los iconos, pasando por el menú de inicio y las entradas existentes

4.3.4 Ejecución de los programas en un icono

Normalmente, cuando se arrancan los programas, éstos se visualizan en una ventana separada; sin embargo, hay ciertos casos en los que tiene sentido que la salida de un programa aparezca directamente sobre un icono del panel. Configurando el icono de esta forma, al ejecutar el programa, sus resultados no aparecerán en una ventana independiente, sino como una parte del panel.

4.3.5 Herramientas del panel

Las herramientas del panel son pequeños programas que visualizan la información de estatus, por ejemplo: Cuando un e-mail ha sido recibido, carga de la CPU, etc. Es similar a la información de estatus que aparece en la parte inferior derecha en el caso de Microsoft Windows.

Estas herramientas pueden ser activadas en el auto arranque

4.3.6 Escritorios virtuales

KDE maneja por defecto cuatro escritorios virtuales pudiéndose acceder a cada uno de ellos a través del KDE panel. El máximo número de escritorios permitidos es ocho, pudiendo ser este número configurado por el centro de control.

4.3.7 Otros aspectos

KDE incorpora otra serie de programas que pueden ser usado en el entorno KDE o de forma independiente bajo UNIX – Linux. Entre los más importantes están:

- Kdehelp: Sistema de ayudas
- Kppp: Configuración del protocolo ppp
- Kmail: Correo electrónico
- Kfm: Gestión de ficheros parecido al explorador de Microsoft Windows
- Kghostview: Visor postscript
- Klix: Procesador de texto

4.4 GNOME

La primera versión del proyecto GNOME nace en el 1997. La versión de 2014 es la actualmente la versión 3.14

GNOME (*GNU Network Object Model Environment*) al igual que KDE ofrece un entorno de escritorio completo y un marco de aplicaciones para desarrollo tan bueno como de fácil uso. A diferencia de KDE, GNOME **no incluye en su proyecto un gestor de ventanas**, proporciona bibliotecas de desarrollo y características de gestión de sesión que nosotros, como usuarios, no vemos. En lo más alto de su estructura está un gestor de ventanas que muestra la apariencia general del escritorio. El gestor de ventana por defecto es **Enlightenment**, pero hay disponibles varias opciones más.

Desde el punto de vista del desarrollador, GNOME define sus interfaces con el resto del mundo mediante tecnología CORBA. Cualquier sistema de desarrollo que pueda comunicarse usando CORBA puede utilizarse para desarrollar aplicaciones compiladas en GNOME.

GNOME está basado en la librería **Gtk (Gimp Toolkit)**. Esta librería fue desarrollada inicialmente como interfase de usuario de Gimp (Programa de tratamiento de imágenes). Actualmente Gtk es realmente software libre y está sujeto a la licencia LGPL.

El principal distribuidor de GNOME en la actualidad es RedHat

4.4.1 Características de GNOME

Los programas de GNOME pueden ser configurados a través de menús.

A diferencia de KDE, GNOME no proporciona distintas configuraciones para diferentes idiomas, sino que éstos se configuran a través de las distintas variables de entorno.

Su gestor natural de ventanas es Enlightenment que ofrece una gestión y configuración verdaderamente amigable

Los principales componentes de GNOME son:

- Gestor de escritorio
- Panel GNOME

4.4.2 Gestor de escritorio

Tiene la responsabilidad de visualización y gestión de los iconos visibles del escritorio. Permite la utilización de drivers que crean iconos para todos los dispositivos, permitiendo a los usuarios montar (mount) y desmontar (umount) dispositivos

Si adicionalmente se utiliza como Enlightenment como gestor de ventanas, permitirá además, utilizar varios escritorios virtuales. Permite la utilización de las llamadas *desktop areas* por cada escritorio, proporcionando un máximo de flexibilidad.

4.4.3 Panel GNOME

Es la unidad central de control. Proporciona el menú de inicio y los applets que muestran sus salidas en pequeñas áreas del panel. Si un panel no es suficiente se pueden crear otros adicionales

El menú de inicio, que es además editable, contiene una colección de los programas más importantes de GNOME (similar al menú de inicio de Microsoft Windows).

Los applets son programas embebidos que son ejecutados en el panel. La configuración más corriente proporciona:

- Pager: Gestiona el tamaño de la imagen para todos los escritorios. Permite cambiar fácilmente entre ventanas y escritorios
- QuickLaunch: Permite la ejecución fácil y rápida de los programas más usado salvando el path de la aplicación a través de la jerarquía de menús (Parecido al favoritos de Microsoft Windows)
- MinComander: Permite la introducción de comandos y su almacén en un histórico
- Reloj

4.4.4 Otros aspectos de GNOME

GNOME, al igual que KDE, proporciona una colección de programas que pueden ser utilizados dentro del entorno GNOME, o ejecutados de forma independiente en Linux. Entre los más usuales destacaremos:

- gnome-help-browser: Sistema de ayudas
- gmc: Gestor de ficheros como el explorador de Microsoft Windows
- gftp: Cliente ftp
- gnome-linuxconf: Configuración del sistema Linux

- **gnorpm**: Gestor de paquetes software para facilitar su instalación
- **GConf**: almacenamiento de configuración del sistema.
- **GNOME VFS**: sistema de ficheros virtual.
- **GNOME Keyring**: sistema de seguridad.
- **GNOME Print**: sistema de impresión de documentos.
- **Gstreamer**: el *framework* o «esqueleto» multimedia para aplicaciones.
- **GTK+**: bibliotecas para desarrollar *interfaces gráficas de usuario*.
 - **ATK**: bibliotecas para ofrecer accesibilidad, por ejemplo, a personas con alguna discapacidad.
- **Pango**: biblioteca para el diseño y renderizado de texto internacional.
- **Cairo**: biblioteca de *render* avanzado de controles de aplicación
- **LibXML**: biblioteca *XML*.
- **Orbit**: un **CORBA ORB** para componentes software.
- **Metacity**: administrador de ventanas.
- **HIG**: investigación y documentación iniciadas por **Sun Microsystems** para aumentar la usabilidad.
- **Nautilus**: administrador de archivos muy completo.
- **GUCharMap**: mapa de caracteres **UNICODE**.
- **Art.**: Sistema Multimedia

PREGUNTAS DE TEST

1. ¿Quién es considerado el desarrollador oficial de Linux?
 - a) Unix.
 - b) Bill Gates.
 - c) Steve Jobs.
 - d) Linus B. Torvalds.
2. ¿Que otro nombre recibe el núcleo del sistema operativo Linux?
 - a) Kernel.
 - b) Shell.
 - c) EXT4.
 - d) Swap.
3. ¿Podemos trabajar en un equipo con Linux sin tenerlo instalado en el disco duro?
 - a) Nunca.
 - b) Si, pero solamente en equipos que cuenten con dispositivos USB.
 - c) Si, siempre y cuando iniciemos el equipo en la opción Probar Ubuntu sin alterar su equipo.
 - d) Solamente en versiones de Linux Server.
4. ¿Cuál es el sistema de archivos recomendado en el que se instalan versiones de Linux modernas?
 - a) EXT4.
 - b) EXT3.
 - c) EXT2.
 - d) Cualquiera de ellos, ya que no hay apenas diferencias.
5. ¿Cómo se llama el gestor de la interfaz gráfica en Linux Ubuntu?
 - a) Gnome.
 - b) KDE.
 - c) Unity.
 - d) Todas son ciertas.
6. ¿Qué tipo de shell se utiliza por defecto en las versiones superiores de Linux Ubuntu?
 - a) Shell job.
 - b) Shell korn.
 - c) C-Shell.
 - d) Bourne Again Shell.
7. ¿Qué opción de inicio de Linux nos permite arrancar el equipo solo con un terminal de trabajo?
 - a) Recovery Console.
 - b) Gnome clásico.
 - c) Ubuntu.
 - d) User defined session.
8. ¿Es lo mismo hibernar el equipo que bloquear la pantalla?
 - a) Si, apenas hay diferencias.
 - b) No, ya que la hibernación hace un volcado a disco de lo que se está ejecutando y el bloqueo de pantalla no.
 - c) No, ya que el bloqueo de pantalla hace una copia a disco de lo que se está ejecutando y la hibernación no.
 - d) Si, si hemos iniciado sesión como super usuario.

PREGUNTAS DE TEST

9. ¿Todos los iconos tienen el mismo menú contextual?
- a) En la mayoría de los casos suelen tener opciones muy parecidas.
 - b) Depende del tipo de icono.
 - c) Si, si trabajamos en entornos KDE.
 - d) Son correctas 1 y 2.
10. ¿A que equivale un enlace en Linux?
- a) A un archivo de información de sistema.
 - b) A un archivo comprimido.
 - c) A un archivo de información de hardware.
 - d) A un acceso directo en Windows.
11. ¿Con que comando podemos apagar el equipo?
- a) Reboot.
 - b) Shutdown.
 - c) Poweroff.
 - d) Son correctas 2 y 3.
12. ¿En qué directorio principal se encuentran los iconos de Linux?
- a) /home.
 - b) /root.
 - c) /usr.
 - d) En ninguno de los anteriores.
13. ¿Cuál de los siguientes caracteres no puede formar parte de un nombre de archivo o directorio?
- a) -.
 - b) _.
 - c) &.
 - d) Ninguno de ellos es correcto.
14. ¿Que representa la barra / en Linux?
- a) La operación de dividir.
 - b) Equipo en red.
 - c) Directorio padre.
 - d) Directorio raíz.
15. ¿Que parte del sistema de archivos almacena un programa que utiliza el sistema para gestionar el resto de las partes del mismo?
- a) Bloque de carga.
 - b) La tabla de inodos.
 - c) Superbloque.
 - d) Zona de datos.
16. ¿Que sistema o sistemas de archivos necesita Linux para funcionar?
- a) Solamente un sistema del tipo ext.
 - b) Solamente un sistema de archivo swap.
 - c) Solamente un sistema de archivos reiser.
 - d) Son correctas 1 y 2.
17. ¿En que directorio tenemos los directorios personales de todos los miembros del sistema?
- a) /etc/dev.
 - b) /root.
 - c) /gnome.

PREGUNTAS DE TEST

- d) /home.
18. ¿Qué significa o a que sustituye [-] cuando utilizamos metacaracteres?
- a) Exclusivamente, dos caracteres.
 - b) Para sustituir un carácter.
 - c) Conjunto de caracteres.
 - d) Rango de caracteres.
19. ¿Cuántos directorios se pueden crear con una sola orden mkdir?
- a) Uno solo.
 - b) Tantos como queramos.
 - c) Tantos como queramos, pero solamente en el directorio actual.
 - d) Solamente dos, uno en el directorio padre y otro en el actual.
20. ¿Cómo se pueden copiar estructuras completas de directorios?
- a) No se puede.
 - b) Con la orden xcopy.
 - c) Con la orden cp utilizando el parámetro -R.
 - d) Con la orden mv y el parámetro -x.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	D
2	A
3	C
4	A
5	D
6	D
7	A
8	B
9	D
10	D
11	D
12	C
13	C
14	D
15	A
16	D
17	D
18	D
19	B
20	C

BIBLIOGRAFÍA BÁSICA

1. Unix System V. A practical guide. Sobell, Mark G. Ed. Addison – Wesley. 1995
2. Manual de administración de Linux
3. Sistemas operativos modernos. Tannenbaum. Andrew. Ed. Prentice Hall. 1993
4. The design of UNIX operating system. Bach, Maurice J. 1986
5. Sistemas operativos monopuesto. Francisco Javier Munoz Lopez. 2013. McGrawHill
6. Administración avanzada de GNU/Linux. Josep Jorba Esteve y Remo Suppi Boldrito. Escuela Universitaria de Informática (Universidad Autónoma de Barcelona, España).
7. Aprenda LINUX como si estuviera en primero. Escuela Superior de Ingenieros Industriales de San Sebastián. UNIVERSIDAD DE NAVARRA
8. MANUAL DE LINUX. Álvaro Alea Fdz.

Nota. La Wikipedia está llena de referencias e imágenes que ilustran muchos de los apartados de este tema. Además, hay muchos libros de texto generales que cubren los sistemas operativos; incluyendo Stallings, Nutt y Tanenbaum.